

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CURSO DE ENGENHARIA DE COMPUTAÇÃO

IGOR LAMOIA QUEIROZ

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO:
Uma abordagem interativa com ambiente de desenvolvimento integrado (ide) para
personalização e execução de comandos definidos pelo usuário

Leopoldina

2026

IGOR LAMOIA QUEIROZ

**INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO:
Uma abordagem interativa com ambiente de desenvolvimento integrado (ide) para
personalização e execução de comandos definidos pelo usuário**

Trabalho de Conclusão de Curso apresentado
ao Curso de Engenharia de Computação do
Centro Federal de Educação Tecnológica de
Minas Gerais, do Campus Leopoldina, como
parte do requisito para obtenção do título de
Bacharel em Engenharia de Computação.

Orientador: Luís Augusto Mattos Mendes

Coorientador: Eduardo Gabriel Reis Miranda

Leopoldina

2026

XXXX Queiroz, Igor Lamoia.

2026 Interpretador Dinâmico para Ensino de Programação : Uma abordagem interativa com ambiente de desenvolvimento integrado (ide) para personalização e execução de comandos definidos pelo usuário / Igor Lamoia Queiroz

Queiroz - Leopoldina : CEFET-MG, 2026.

ix, 78 f. : il.

Orientador: Luís Augusto Mattos Mendes

Coorientador: Eduardo Gabriel Reis Miranda

TCC (Graduação) – Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina. Engenharia de Computação.

1. Interpretador dinâmico. 2. Ensino de programação. 3. Compiladores. 4. Ambiente de Desenvolvimento Integrado. I. Mendes, Luís Augusto Mattos. II. Miranda, Eduardo Gabriel Reis. III. Interpretador Dinâmico para Ensino de Programação.

CDU XXX.XX

IGOR LAMOIA QUEIROZ

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO : Uma abordagem interativa com ambiente de desenvolvimento integrado (ide) para personalização e execução de comandos definidos pelo usuário

Trabalho de conclusão de curso apresentado ao Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, do Campus Leopoldina, como parte do requisito para obtenção do título de Bacharel em Engenharia de Computação.

Banca Examinadora:

Prof. **Luís** Augusto Mattos Mendes- Orientador
CEFET-MG

Prof. Dsc. Eduardo Gabriel Reis Miranda- Coorientador
CEFET-MG

Prof. Me. Matheus Ávila Moreira de Paula
CEFET-MG

Prof. Dsc. Gabriella Castro Barbosa Costa
CEFET-MG

(Assinaturas Digitais)

Leopoldina
2026

AGRADECIMENTOS

Agradeço à minha família pelo apoio incondicional, incentivo e paciência ao longo desta jornada. Sem vocês, nada disso seria possível. Meu sincero agradecimento também ao meu orientador, Luís Augusto Mattos Mendes, e ao meu coorientador, Eduardo Gabriel Reis Miranda, pela orientação, dedicação e ensinamentos valiosos que tornaram este trabalho possível. A todos que, de alguma forma, contribuíram para esta conquista, meu muito obrigado.

RESUMO

Ensinar programação apresenta diversos desafios, em parte devido ao uso de linguagens cuja escrita é definida por sintaxe fixa e palavras-chave reservadas. Isso pode dificultar a assimilação por parte dos estudantes que, mesmo antes de compreender a estrutura dos algoritmos, precisam memorizar termos em outro idioma e conjuntos de regras rígidas. **Este trabalho** propõe uma plataforma educacional com um Ambiente de Desenvolvimento Integrado (IDE) conectado a um interpretador dinâmico que permite aos usuários personalizar a sintaxe da linguagem, os comandos e as regras **estruturais**. O projeto é desenvolvido por meio de revisão de literatura, projeto da arquitetura do sistema, implementação de uma IDE web e de um interpretador (incluindo análise léxica e sintática) e integração com execução em tempo real no navegador. Espera-se que a plataforma contribua para o ensino de programação ao reduzir barreiras linguísticas, favorecer a compreensão de conceitos fundamentais e estimular o engajamento dos estudantes por meio da experimentação com diferentes formas de expressão algorítmica. Ao permitir a definição flexível de comandos e sintaxe, a plataforma incentiva a experimentação, ajudando os alunos a focarem no raciocínio algorítmico em vez de memorizar estruturas rígidas de linguagem.

Palavras-chave: Interpretador Dinâmico; Educação em Programação; Ambiente Educacional Web; IDE Educacional; Personalização de Linguagens.

ABSTRACT

Teaching programming presents several challenges, partly due to the use of languages whose code structure is defined by fixed syntax and reserved keywords. This can hinder student learning, as beginners often need to memorize foreign-language terms and rigid sets of rules before fully understanding algorithmic concepts. This work proposes an educational platform featuring an Integrated Development Environment (IDE) connected to a dynamic interpreter that allows users to customize language syntax, commands, and structural rules. The project is developed through a literature review, system architecture design, implementation of a web-based IDE and interpreter (including lexical and syntactic analysis), and integration with real-time execution in the browser. The platform is expected to contribute to programming education by reducing linguistic barriers, supporting the understanding of fundamental concepts, and increasing student engagement through experimentation with different forms of algorithmic expression. By enabling flexible definition of commands and syntax, the proposed approach encourages experimentation and helps students focus on algorithmic reasoning rather than memorizing rigid language structures.

Keywords: Dynamic Interpreter; Programming Education; Web-Based Educational Environment; Educational IDE; Language Customization.

LISTA DE ILUSTRAÇÕES

Figura 2.1 –Arquitetura de um compilador moderno.	16
Figura 2.2 –Exemplificação de agrupamento de caracteres em lexemas.	19
Figura 4.1 –Arquitetura geral da plataforma e fluxos de comunicação entre as camadas. .	45
Figura 4.2 –Página inicial da plataforma em suas duas variações de tema.	60
Figura 4.3 –Painel principal do aluno, com acesso às turmas, listas e exercícios.	61
Figura 4.4 –Composições da experiência do aluno: visão da turma e visão da lista de exercícios.	62
Figura 4.5 –Painel principal do professor.	62
Figura 4.6 –Composições da experiência do professor: visão da turma e visão das listas associadas.	63
Figura 4.7 –Fluxo de criação e gestão de listas de exercícios pelo professor.	65
Figura 4.8 –Fluxo de criação e acompanhamento dos enunciados pelo professor.	67
Figura 4.9 –Etapas 1 e 2 do assistente de personalização da linguagem dinâmica.	68
Figura 4.10 –Etapa 3 do assistente, dedicada às palavras-chave de controle de fluxo. . . .	69
Figura 4.11 –Etapa 4 do assistente, dedicada à substituição de operadores por palavras. . .	70
Figura 4.12 –Etapa 5: personalização dos literais booleanos.	71
Figura 4.13 –Etapa 6 do assistente, dedicada aos delimitadores, ao terminador de instrução e à pré-visualização da linguagem personalizada.	72

LISTA DE TABELAS

Tabela 2.1 – Comparativo entre as ferramentas analisadas e a plataforma proposta.	38
Tabela 3.1 – Cronograma de desenvolvimento do trabalho de conclusão de curso.	41

SUMÁRIO

1	INTRODUÇÃO	11
1.1	Justificativa	12
1.2	Objetivos	13
1.2.1	Objetivo Geral	13
1.2.2	Objetivos Específicos	13
1.3	Estrutura do Trabalho	14
2	REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS . . .	15
2.1	Compiladores	15
2.1.1	Compiladores e seus Fundamentos	15
2.1.2	O Modelo de Análise e Síntese (Fases da Compilação)	16
2.1.3	A Análise Léxica e o Reconhecimento de <i>Tokens</i>	18
2.1.4	Estratégias de Análise Sintática: <i>Top-Down</i> e <i>Bottom-Up</i>	19
2.1.5	Análise Semântica e Verificação de Tipos	21
2.1.6	Geração de Código Intermediário	22
2.1.7	Otimização de Código e Eficiência Computacional	24
2.1.8	Geração de Código Objeto e Mapeamento de Arquitetura	25
2.1.9	Engenharia de Software	26
2.1.9.1	Gestão de Tarefas e Metodologias Ágeis	27
2.1.9.2	Testes Automatizados e <i>Test-Driven Development</i> (TDD)	28
2.1.9.3	Arquitetura Limpa e Separação de Responsabilidades	29
2.1.9.4	Qualidade de Código e <i>Clean Code</i>	30
2.1.9.5	O Framework Next.js e sua Arquitetura	31
2.1.9.6	Design de Experiência do Usuário (UX) e Interface do Usuário (UI)	32
2.2	Softwares Educativos e Tecnologias de Apoio à Aprendizagem	33
2.3	Trabalhos Relacionados	34
2.3.1	Portugol Studio e Portugol Webstudio	35
2.3.2	Laila: Gerador de Analisadores Léxicos e Sintáticos Online	35
2.3.3	Quorum: Linguagem de Programação Baseada em Evidências	36
2.3.4	chibicc: Uma Abordagem Incremental para o Ensino de Compiladores . . .	37
2.3.5	Beecrowd: Prática de Algoritmos e Avaliação Automatizada	37

2.3.6	Análise Crítica e o Diferencial da Solução Proposta	38
3	PROPOSTA	40
4	METODOLOGIA	43
4.1	Modelagem da Arquitetura da Plataforma	43
4.2	Interpretador	45
4.3	Customização Dinâmica de Tokens e Sintaxe	46
4.4	Edição de Código e Sistema de Arquivos Virtual	53
4.5	Execução das Análises Léxica e Sintática a partir da IDE	53
4.6	Execução do Código Intermediário e <i>Feedback</i> no Terminal	56
4.7	Sistema de Correção e Avaliação Automatizada	57
4.8	Acessibilidade, Responsividade e Finalização de Fluxos	58
4.9	Apresentação das Telas da Plataforma	59
4.9.1	Fluxo de Personalização da Linguagem Dinâmica	68
4.10	Estratégia de Testes Automatizados	73
	REFERÊNCIAS	74
A	APÊNDICE A – PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES	77
A.1	Atividades Desenvolvidas em Regime Colaborativo	77
A.2	Eixos de Aprofundamento Individual	77
A.2.1	Igor Lamoia Queiroz (este documento)	77
A.2.2	Victor de Souza Vilela da Silva (documento complementar)	78

1 INTRODUÇÃO

O presente Trabalho de Conclusão de Curso foi desenvolvido em cooperação técnico-científica, em coautoria com o acadêmico Victor de Souza Vilela da Silva, do Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina. Esta pesquisa faz parte de um estudo complementar e interdependente intitulado *Interpretador Dinâmico para Ensino de Programação: Implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web*, de autoria do referido coautor e submetido ao mesmo curso desta Instituição. A divisão das atividades comuns e dos eixos individuais de aprofundamento é descrita no Apêndice A.

Embora ambos os documentos compartilhem a mesma base teórica geral e o mesmo objeto de estudo, isto é, uma plataforma educacional web formada por um interpretador dinâmico integrado a uma IDE com gramática configurável pelo usuário, as abordagens diferem em escopo e ênfase metodológica. Enquanto o documento complementar se concentra no núcleo de tradução, examinando as fases de análise léxica, análise sintática, geração de código intermediário e execução pelo interpretador, este volume trata do Ambiente de Desenvolvimento Integrado: sua arquitetura *front-end*, a criação e implementação do assistente de personalização da linguagem, a experiência de uso da plataforma e os procedimentos de coleta de *feedback* junto aos estudantes. Para uma compreensão completa do sistema proposto, recomenda-se a leitura dos dois documentos.

~~As reflexões que abrem este capítulo, bem como as considerações sobre as dificuldades enfrentadas por estudantes em seu primeiro contato com a programação, apoiam-se principalmente nos estudos de Stefik e Siebert (2013) e Ferreira, Silva e Ribeiro (2025).~~

A familiaridade mínima com a programação deixou de ser uma exigência restrita aos cursos de Ciência da Computação e da Engenharia de Computação. Áreas tão distintas quanto economia, biologia, ciências sociais e jornalismo passaram a usar a manipulação programática de dados como ferramenta comum de trabalho, e essa presença em várias áreas tende a crescer. Em contraste com esse cenário, as disciplinas de iniciação à programação seguem registrando taxas elevadas de reprovação e de evasão (SOUZA; BATISTA; BARBOSA, 2016; SILVA et al., 2020). Para enfrentar esse problema, é necessário compreender em que ponto da aprendizagem o estudante costuma encontrar maior resistência.

Observa-se, com frequência, que a barreira não está no raciocínio algorítmico em si.

Estudantes que conseguem descrever, em linguagem natural, a sequência de passos necessária para resolver um problema encontram dificuldade, no momento de levar esse raciocínio para o computador, por causa da rigidez das linguagens de programação tradicionais. Pontos e vírgulas omitidos, chaves desbalanceadas, palavras reservadas escritas em idioma estrangeiro e mensagens de erro difíceis de entender formam um conjunto de pequenas exigências que, embora secundárias do ponto de vista lógico, consomem parte considerável do tempo e da energia cognitiva do **iniciante**. A frustração gerada por esse atrito tende a se acumular nas primeiras semanas de curso, justamente o período em que a percepção de progresso é mais decisiva para a continuidade do **estudo**.

A pergunta que orienta esta pesquisa nasce dessa observação: **é possível criar uma plataforma de programação que permita ao próprio estudante redefinir o vocabulário e as regras estruturais da linguagem com que aprende, de modo a deslocar o foco da forma para o conteúdo do raciocínio computacional?**

A hipótese defendida ao longo do trabalho responde afirmativamente a essa pergunta. Parte-se da hipótese de que, ao permitir que o estudante escolha quais palavras servirão como comandos, como os blocos serão delimitados, como os operadores serão escritos e se o terminador de instrução será obrigatório ou opcional, **reduz-se de forma significativa a carga cognitiva exigida no aprendizado simultâneo da lógica e da forma**. O exercício de adaptar a linguagem ao próprio repertório transforma a sintaxe, antes vivida como obstáculo, em objeto consciente de escolha pedagógica.

1.1 Justificativa

A motivação para desenvolver este projeto baseia-se em uma constatação obtida a partir da análise das ferramentas educacionais hoje disponíveis: embora diversas plataformas se proponham a facilitar o ensino de programação, a grande maioria opera com uma gramática fixa, definida previamente por seus responsáveis. Linguagens didáticas em português, juízes online voltados à prática contínua e ambientes de programação visual têm **méritos próprios**; nenhum deles, contudo, oferece ao usuário final a possibilidade de reescrever as regras estruturais que definem como o pensamento é convertido em código.

A construção de uma ferramenta que inverta essa relação, adaptando a linguagem ao estudante em vez de exigir que o estudante se adapte à linguagem, é assumida aqui como uma abordagem ainda pouco explorada no campo da informática educacional: entre as ferramentas

educacionais de uso corrente predominam gramáticas fixas, definidas previamente por seus desenvolvedores, e a possibilidade de o próprio estudante redefinir as regras estruturais da linguagem permanece pouco difundida, conforme evidencia a análise comparativa apresentada no referencial teórico. Diferentemente de iniciativas voltadas apenas ao núcleo de tradução ou apenas à interface, este trabalho integra essas duas frentes: um interpretador capaz de aceitar gramáticas configuradas pelo usuário e uma IDE web cuja experiência de uso oferece, ao mesmo tempo, **conforto operacional** próximo ao de ferramentas profissionais e **apoio pedagógico** adequado ao público iniciante.

1.2 Objetivos

A seguir são apresentados o objetivo geral que orienta a investigação e os objetivos específicos que viabilizam o seu alcance.

1.2.1 Objetivo Geral

Construir uma plataforma educacional formada por uma IDE web e por um interpretador dinâmico que aceite, em tempo de execução, regras gramaticais definidas pelo próprio usuário, a fim de oferecer suporte ao ensino e à aprendizagem de algoritmos por estudantes em estágio introdutório.

1.2.2 Objetivos Específicos

Para que o objetivo geral seja efetivamente alcançado, as seguintes metas operacionais foram fixadas:

- **Estudar**, de forma crítica, os fundamentos de construção de compiladores e as ferramentas educacionais disponíveis na literatura;
- **Participar do projeto** e da integração do interpretador, contemplando a execução das análises léxica e sintática e do código intermediário a partir da IDE;
- **Projetar a IDE web**, com recursos para personalização da linguagem, execução de código no próprio ambiente e apresentação de *feedback* ao usuário;
- Validar a plataforma por meio de **estudos de caso** em ambiente educacional, observando aspectos de usabilidade, de viabilidade técnica e de potencial pedagógico.

1.3 Estrutura do Trabalho

O restante deste trabalho está organizado da seguinte maneira. O Capítulo 2 apresenta o referencial teórico que sustenta o projeto, reunindo os fundamentos de construção de compiladores, as práticas associadas ao desenvolvimento de aplicações web e as iniciativas educacionais relacionadas. O Capítulo 3, por sua vez, expõe a proposta do trabalho e o cronograma seguido em suas duas fases. Por fim, o Capítulo 4 descreve a modelagem realizada e a implementação consolidada até o encerramento desta etapa, contemplando a arquitetura da plataforma, a personalização dinâmica da linguagem, a integração com o interpretador e a estratégia de testes adotada.

2 REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS

Este capítulo apresenta o referencial teórico que fundamenta o desenvolvimento da plataforma. A Seção 2.1 discute a construção de compiladores e interpretadores, percorrendo as fases de análise léxica, sintática e semântica, a geração de código intermediário e objeto e as estratégias de otimização, além das metodologias ágeis de desenvolvimento, da arquitetura do *framework* Next.js, dos princípios de Experiência do Usuário (UX) e Interface do Usuário (UI) e das práticas de qualidade de código. Em seguida, são apresentados os softwares educativos e as tecnologias de apoio à aprendizagem e, por fim, os trabalhos relacionados, cuja análise crítica fundamenta o diferencial da solução proposta.

2.1 Compiladores

Esta seção percorre os fundamentos de compiladores que embasam o núcleo de tradução implementado neste trabalho, desde a definição de compilador e de suas fases até a geração de código intermediário.

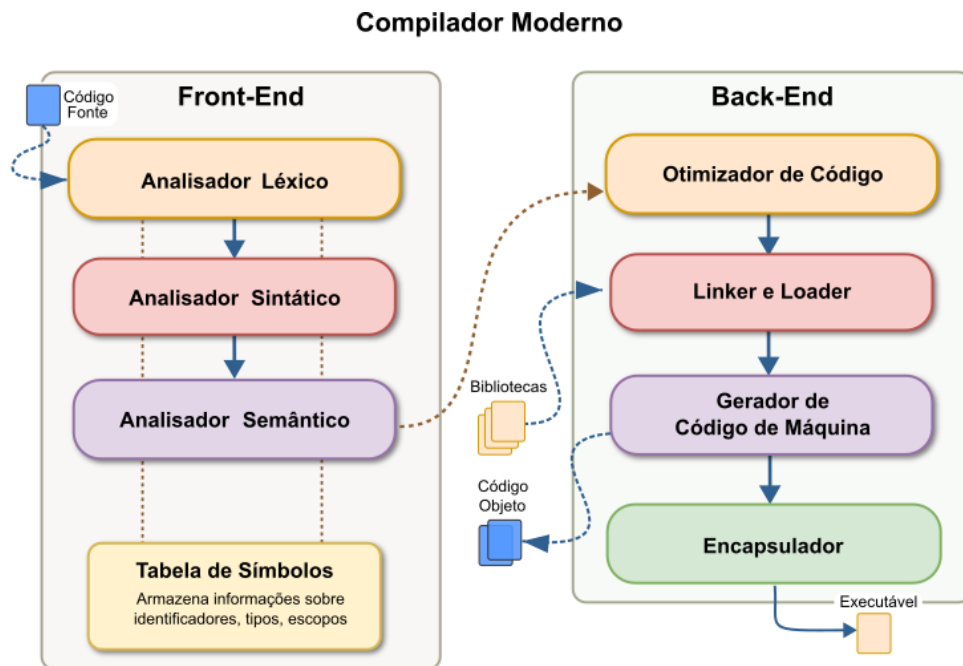
2.1.1 Compiladores e seus Fundamentos

~~Esta subseção usa como base a obra clássica de Aho, Sethi e Ullman (1995), amplamente reconhecida na literatura de Ciência da Computação como uma das principais referências teóricas no projeto de compiladores. As definições de análise, síntese e estrutura do processo de compilação apresentadas a seguir seguem as ideias desses autores.~~

~~Em termos práticos,~~ um compilador é um programa cuja função é ler um código-fonte, escrito em uma linguagem específica, e traduzi-lo para um programa equivalente em uma linguagem-alvo. No modelo moderno de compilação, esse processo é estruturado em duas etapas principais e complementares: a análise e a síntese. A etapa de análise, comumente chamada de *front-end*, divide o programa original em suas partes e gera uma representação intermediária do código. Por outro lado, a etapa de síntese, conhecida como *back-end*, utiliza essa representação intermediária para construir e formatar o programa final desejado (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para facilitar a compreensão dessa separação lógica e visualizar a evolução da transformação do código, observe a Figura 2.1.

Figura 2.1 – Arquitetura de um compilador moderno.



Fonte: Alcantara (2024).

De modo conceitual, a operação de um compilador ocorre por meio de fases sequenciais, nas quais cada etapa tem o papel de converter o programa de um nível de abstração para outro. O núcleo de análise é formado pelas fases iniciais, que englobam as análises léxica, sintática e semântica. A partir desse ponto, o fluxo do compilador avança para as etapas de síntese, que englobam a geração do código intermediário, a otimização desse código e, por fim, a geração do código-alvo (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A tabela de símbolos é gerenciada ao longo de todas essas fases. Trata-se de uma estrutura de dados muito importante para o compilador, pois mapeia todos os identificadores usados no programa fonte. Ao coletar e armazenar informações sobre os atributos de cada identificador, essa tabela permite recuperar escopos rapidamente e realizar a validação semântica durante a tradução (AHO; SETHI; ULLMAN, 1995; PRICE; TOSCANI, 2001).

2.1.2 O Modelo de Análise e Síntese (Fases da Compilação)

~~A fundamentação teórica desta seção baseia-se nas obras de referência de Aho, Sethi e Ullman (1995), Price e Toscani (2001) e Cooper e Torczon (2012), que apresentam os princípios estruturais do processo de compilação.~~

De modo geral, a tradução é normalmente dividida em duas etapas principais: a análise e a síntese. No estágio de análise (*front-end*), o foco está em interpretar o programa

original, dividindo-o em componentes menores para gerar uma abstração chamada representação intermediária. Por outro lado, o estágio de síntese (*back-end*) converte essa representação no programa final pretendido. Essa distinção arquitetural entre *front-end* e *back-end* funciona como uma camada de abstração que simplifica a portabilidade, permitindo que o compilador suporte diversas linguagens e diferentes plataformas de hardware com maior agilidade (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para lidar com a complexidade da tradução de linguagens de alto nível, o fluxo operacional é dividido em unidades lógicas menores, chamadas de fases. Cada fase é responsável por uma transformação específica na representação do código. Na prática, essas fases podem ser agrupadas em "passagens" (*passes*), estratégias que percorrem o código para otimizar a gerência de memória e o desempenho do sistema (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

As principais etapas que integram um compilador são detalhadas a seguir:

- **Análise Léxica (*Scanner*):** É a primeira etapa da compilação e processa o código-fonte caractere a caractere. Sua função é converter o fluxo de texto em uma sequência de unidades lógicas conhecidas como *tokens*, removendo elementos irrelevantes para o processamento, como comentários e espaços redundantes (COOPER; TORCZON, 2012; NYSTROM, 2021).
- **Análise Sintática (*Parser*):** Utiliza a sequência de *tokens* para validar a conformidade gramatical do programa em relação às regras da linguagem. O *parser* organiza esses elementos em estruturas hierárquicas (as árvores sintáticas ou de derivação), empregando métodos descendentes (*top-down*) ou redutivos (*bottom-up*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Análise Semântica:** Esta fase vai além da forma gramatical para garantir que as instruções tenham coerência lógica e operacional. Uma atividade importante aqui é a checagem de tipos (*type checking*), que valida se a interação entre operadores, variáveis e funções respeita as definições da linguagem (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Geração de Código Intermediário:** Concluídas as análises, o sistema produz uma versão do código que é independente da arquitetura final. O "código de três endereços" é um formato recorrente nesta etapa, servindo para reduzir a complexidade entre a abstração

do alto nível e o rigor das instruções de máquina (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

- **Otimização de Código:** Dedicar-se ao refinamento do código intermediário, buscando torná-lo mais eficiente. O objetivo principal é reduzir o tempo de processamento e a ocupação de memória sem alterar o comportamento funcional do programa (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Geração de Código Objeto:** Marca o encerramento da fase de síntese, na qual a representação intermediária é mapeada para a linguagem alvo (seja *assembly* ou código de máquina). Esta etapa exige precisão na escolha das instruções nativas e cuidado na gestão dos registradores e endereços de memória do processador (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Durante todo o ciclo, dois componentes dão suporte contínuo: o gerenciador da Tabela de Símbolos e o Tratador de Erros. A tabela de símbolos centraliza o registro de identificadores e seus metadados (como tipo e escopo), garantindo acesso rápido às informações em qualquer fase. Já o módulo de erros monitora falhas léxicas, sintáticas e lógicas, aplicando rotinas de recuperação que permitem ao compilador prosseguir com a análise e identificar o maior número possível de inconsistências em uma única execução (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

2.1.3 A Análise Léxica e o Reconhecimento de *Tokens*

~~As bases teóricas da etapa de análise léxica, exploradas nesta subseção, apoiam-se nos estudos de Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).~~

Conhecida tecnicamente como *scanner*, a análise léxica representa o estágio inicial do pipeline de compilação. Sua principal responsabilidade é percorrer o código-fonte caractere por caractere, processando essa entrada bruta para agrupar elementos em unidades com significado para a gramática da linguagem, chamadas de lexemas (conforme ilustrado na Figura 2.2) (COOPER; TORCZON, 2012; NYSTROM, 2021).

Ao mesmo tempo em que organiza esses elementos, o analisador léxico realiza uma "limpeza" no fluxo de dados. Ele identifica e descarta componentes que, embora úteis ao programador, são irrelevantes para a tradução lógica, como comentários, tabulações e espaços em branco excedentes (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Figura 2.2 – Exemplificação de agrupamento de caracteres em lexemas.



Fonte: Nystrom (2021, p. 43).

É importante, contudo, não confundir os conceitos de *lexema* e *token*. O *lexema* refere-se à sequência literal de caracteres encontrada no texto original (a *string* propriamente dita, como "if", "soma" ou "42"). Já o *token* é um objeto abstrato produzido pelo compilador para representar esse *lexema*. O *token* carrega a categoria sintática da unidade (identificador, operador, palavra-chave) e, frequentemente, metadados adicionais, como o valor convertido e as coordenadas de linha e coluna, importantes para o rastreamento de erros em fases posteriores (COOPER; TORCZON, 2012; NYSTROM, 2021).

Para que o reconhecimento desses padrões ocorra de forma rigorosa, a micro-sintaxe da linguagem é definida por meio de expressões regulares. Esse modelo matemático permite descrever, de maneira concisa, as regras de formação de cada categoria de *token*, definindo, por exemplo, quais combinações de símbolos formam um número real ou um nome de variável válido dentro daquela linguagem (PRICE; TOSCANI, 2001; COOPER; TORCZON, 2012).

Contudo, expressões regulares são ferramentas de especificação, não de execução. Para que o computador processe essas regras, elas são convertidas em autômatos finitos. Um autômato funciona como uma máquina de estados: ele consome a entrada caractere por caractere, transitando entre estados lógicos até atingir um "estado de aceitação". Quando isso ocorre, o *lexema* é validado e o *token* correspondente é enviado para o analisador sintático (PRICE; TOSCANI, 2001; COOPER; TORCZON, 2012).

A implementação baseada em autômatos garante que a análise léxica seja eficiente, operando em tempo linear em relação ao volume de código. Na prática, desenvolvedores utilizam ferramentas de geração automática, como o *Lex*, que transformam definições baseadas em expressões regulares diretamente em código otimizado de autômatos finitos determinísticos (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

2.1.4 Estratégias de Análise Sintática: *Top-Down* e *Bottom-Up*

~~Os conceitos e definições apresentados a seguir têm como base teórica os trabalhos de Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Price e Toscani (2001) e Nystrom (2021).~~

A etapa de análise sintática, comumente chamada de *parsing*, tem a tarefa de validar se a sequência de *tokens* gerada previamente está de acordo com as regras gramaticais da linguagem de origem. Para processar essa estrutura e gerar a árvore de derivação, o sistema utiliza algoritmos baseados em duas abordagens principais: a análise descendente (*top-down*), que atua da raiz para as folhas, e a análise ascendente (*bottom-up*), que opera no sentido inverso (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

I) Análise Sintática Descendente (*Top-Down*): Esta abordagem busca reconstruir a árvore sintática partindo do símbolo inicial (raiz) até alcançar os elementos terminais (folhas) que compõem o código de entrada. Durante esse fluxo, o analisador expande de forma sistemática o símbolo não-terminal posicionado mais à esquerda, utilizando as regras de produção para realizar o que se conhece tecnicamente como derivação à esquerda (*leftmost derivation*) (AHO; SETHI; ULLMAN, 1995; PRICE; TOSCANI, 2001).

Dentro dessa categoria, destacam-se os métodos recursivos descendentes e os preditivos baseados em tabelas, como o LL(1). O uso de analisadores por descida recursiva é frequente em projetos reais devido à sua legibilidade, simplicidade de implementação manual e eficiência no tratamento e diagnóstico de erros. Entretanto, para garantir um processamento linear e evitar o *backtracking* (retrocesso), a gramática precisa ser cuidadosamente preparada: ela não deve apresentar recursão à esquerda e precisa ser devidamente fatorada. Isso assegura que o analisador consiga decidir qual regra aplicar consultando apenas o próximo *token* da entrada (COOPER; TORCZON, 2012; NYSTROM, 2021).

II) Análise Sintática Ascendente ou Redutiva (*Bottom-Up*): Diferente do modelo anterior, a análise ascendente inicia o reconhecimento a partir dos *tokens* individuais (folhas) e progride estruturalmente até atingir o símbolo inicial da gramática (raiz) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

O termo “reductiva” caracteriza bem o funcionamento deste algoritmo: o analisador identifica padrões na pilha que correspondem ao lado direito de uma regra gramatical (denominados *handles*) e os substitui (reduz) pelo respectivo símbolo não-terminal do lado esquerdo. Esse procedimento resulta em uma derivação à direita realizada de forma reversa (AHO; SETHI; ULLMAN, 1995; PRICE; TOSCANI, 2001).

Os analisadores da família LR (como SLR, LALR e o LR Canônico) representam o que há de mais robusto nesta categoria, operando por meio de um sistema de empilhamento e redução (*shift-reduce*) controlado por tabelas de estados. Uma vantagem significativa sobre a técnica

descendente é a flexibilidade, pois os métodos *bottom-up* lidam nativamente com recursão à esquerda e suportam quase todas as construções de gramáticas livres de contexto (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Contudo, a criação manual das tabelas de transição para esses analisadores envolve alta complexidade matemática. Por esse motivo, a prática comum não envolve a codificação manual do *parser*, mas sim o uso de ferramentas de geração automática, como o YACC, que traduzem a gramática formal diretamente em código funcional (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

2.1.5 Análise Semântica e Verificação de Tipos

Após a verificação gramatical feita pela análise sintática, o compilador entra na fase de análise semântica. Segundo Price e Toscani (2001), esta etapa tem como objetivo principal garantir que as estruturas e construções do programa, embora já validadas sintaticamente, façam sentido lógico para a posterior execução do código. Enquanto o *parser* (analisador sintático) assegura que a forma do código está correta, o analisador semântico vai além, inspecionando o que as peças do programa realmente significam de forma combinada (NYSTROM, 2021). Nesse cenário, uma de suas atividades fundamentais é a resolução de associações, na qual o compilador verifica todas as variáveis mencionadas e determina exatamente a qual declaração de escopo cada identificador se refere (NYSTROM, 2021).

Outro componente central da análise semântica é a verificação de tipos (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012). Nesta etapa, o compilador utiliza a estrutura hierárquica gerada previamente (a árvore de derivação) em conjunto com as informações coletadas na tabela de símbolos para checar se cada operador da linguagem está recebendo os operandos que lhe são legalmente permitidos (AHO; SETHI; ULLMAN, 1995). A análise semântica atua inferindo tipos para as expressões e detectando situações nas quais os valores são interpretados incorretamente (COOPER; TORCZON, 2012). Quando permitido pela especificação da linguagem, é também nesta fase que o compilador realiza conversões de tipo embutidas (coerções), como a transformação implícita de um valor inteiro para ponto flutuante antes de realizar uma adição entre tipos mistos (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Os erros semânticos, por sua vez, são falhas peculiares, pois ocorrem em programas que podem possuir uma estrutura gramatical perfeitamente bem-formada, mas que violam as

regras de significado, consistência ou contexto da linguagem de programação (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012). O tratamento de erros semânticos é focado em capturar inconsistências que incluem, principalmente:

- **Incompatibilidade de Tipos de Operandos:** Ocorre quando um operador é aplicado a um ou mais operandos cujos tipos de dados são conflitantes e não possuem regras de conversão implícita ou explícita (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012). Um exemplo prático na maioria das linguagens de programação fortemente tipadas é a tentativa inválida de se aplicar um operador aritmético binário a uma *string* (texto) e a um booleano, ou tentar adicionar um tipo de precisão dupla a um tipo complexo de forma irregular (COOPER; TORCZON, 2012).
- **Uso de Variáveis Não Declaradas ou Falhas de Escopo:** Este erro se manifesta quando o código faz referência a uma variável, função ou objeto que não foi previamente definido no programa, ou cuja declaração se encontra em um escopo léxico distinto e inacessível naquele momento (COOPER; TORCZON, 2012; NYSTROM, 2021). A utilização de uma variável local fora de seu bloco restrito, por exemplo, acarretará uma falha semântica de identificador não reconhecido ou indefinido (NYSTROM, 2021).
- **Inconsistência de Aridade e Estruturas:** Acontece comumente em construções que exigem conformidade de números, como o número incorreto de argumentos (COOPER; TORCZON, 2012). Exemplos incluem uma invocação de função ou chamada de procedimento na qual a quantidade de parâmetros passados difere da quantidade estipulada em sua definição original (COOPER; TORCZON, 2012; NYSTROM, 2021). Igualmente, configura-se um erro semântico referenciar um *array* informando um número de dimensões diferente do limite (ou *rank*) estipulado em sua declaração (COOPER; TORCZON, 2012).

2.1.6 Geração de Código Intermediário

~~Os fundamentos teóricos que norteiam a produção de código intermediário e o processo de tradução dirigida pela sintaxe aqui expostos baseiam-se nas contribuições de Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).~~

A fase de geração de código intermediário funciona como um mediador essencial entre a análise (*front-end*) e a síntese (*back-end*) de um sistema de compilação. Uma vez validadas as estruturas e a semântica do programa original, o compilador elabora uma representação

interna explícita, projetada para ser agnóstica tanto em relação à linguagem de origem quanto à arquitetura de destino. Essa estratégia de tradução em múltiplos estágios é fundamental para decompor a complexidade de converter abstrações de alto nível em comandos de baixo nível de forma controlada. Além disso, essa camada de isolamento facilita a aplicação de técnicas de otimização e potencializa a portabilidade, visto que uma única representação interna pode ser mapeada para diferentes plataformas de *hardware* (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Dentre os formatos de representação linear mais difundidos na literatura técnica, destaca-se o código de três endereços, que opera como um tipo de linguagem de montagem (*assembly*) abstrata e universal. Nessa convenção, operações lógicas e aritméticas complexas são fragmentadas em instruções elementares contendo, no máximo, três operandos. A distinção crucial entre este formato e o código objeto final é que a representação intermediária ignora restrições físicas, como a gestão específica de registradores e o mapeamento direto de endereços de memória (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para exemplificar o conceito, considere uma expressão de atribuição como $A := X + Y * Z$. O compilador desmembra essa lógica em instruções de *pseudo-assembly* simplificadas, utilizando variáveis temporárias para armazenar resultados parciais. O fluxo resultante em código de três endereços seguiria a sequência (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012):

1. $T_1 := Y * Z$
2. $T_2 := X + T_1$
3. $A := T_2$

A automação desse processo de conversão é viabilizada pela técnica de tradução dirigida pela sintaxe. Esse método expande o formalismo das gramáticas livres de contexto, permitindo que atributos sejam associados aos símbolos gramaticais e que ações semânticas específicas sejam vinculadas às regras de produção da linguagem (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Na implementação prática, a emissão do código intermediário ocorre de forma síncrona com a análise sintática. No momento em que o *parser* identifica uma construção gramatical legítima, ele dispara a ação semântica correspondente. Esse procedimento processa os atributos

herdados ou sintetizados dos nós da árvore sintática, permitindo que os fragmentos de código sejam gerados de maneira ordenada e coerente com a lógica original do programa (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

2.1.7 Otimização de Código e Eficiência Computacional

~~A exploração dos princípios de otimização de código aqui delineada encontra seu lastro teórico nos tratados fundamentais de Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Price e Toscani (2001) e Nystrom (2021).~~

Posicionada estrategicamente como um estágio de refinamento no *pipeline* de compilação, a otimização busca elevar a qualidade da representação interna do software. Esse aprimoramento vai além do ganho de velocidade e da economia de memória, abrangendo também a redução do consumo energético, um requisito crítico na computação contemporânea. Durante esta fase, o compilador atua substituindo construções genéricas por equivalentes semânticos mais ágeis. Um exemplo notório é o dobramento de constantes (*constant folding*), que antecipa operações entre valores fixos ainda em tempo de compilação, eliminando o custo desse processamento durante a execução do programa (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A legitimidade de qualquer transformação de código está condicionada a dois critérios imperativos: a preservação semântica (segurança) e a eficácia computacional (rentabilidade). O pilar da segurança determina que a lógica original do algoritmo deve permanecer inalterada, assegurando que a otimização não introduza efeitos colaterais. Já a rentabilidade exige que a alteração proporcione uma melhoria de desempenho real e mensurável, justificando o esforço computacional e o tempo adicional dedicados pelo compilador ao processo (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Quanto à abrangência das intervenções, a otimização é classificada em quatro níveis de profundidade:

- **Otimização Local:** Restringe-se à análise de blocos básicos, isto é, sequências de instruções sem desvios de controle. Neste nível, utilizam-se frequentemente Grafos Acíclicos Dirigidos (GADs) para detectar e remover subexpressões redundantes e gerir variáveis locais de forma eficiente (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Otimização Regional:** Expande o foco para conjuntos de blocos básicos, concentrando

esforços prioritariamente em laços de repetição (*loops*). Dado que a maior parte da carga de processamento ocorre nestes ciclos, melhorias como o deslocamento de código invariante geram impactos substanciais na performance global (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

- **Otimização Global (Intraprocedural):** Analisa o contexto integral de uma função ou procedimento. Por meio de estudos minuciosos do fluxo de dados, o compilador identifica oportunidades de simplificação que seriam invisíveis sob uma inspeção puramente local ou limitada (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Otimização Interprocedural:** Atua no nível mais macro, transpondo as fronteiras entre diferentes módulos e sub-rotinas. Este escopo permite manobras agressivas como o *inlining*, onde o corpo de uma função é integrado diretamente ao ponto de chamada, eliminando a sobrecarga (*overhead*) associada ao desvio de execução e à gestão de pilhas de ativação (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Na prática, a maioria das rotinas de otimização é dividida em duas etapas: uma fase de análise, dedicada ao mapeamento de dependências e do fluxo de controle, seguida por uma fase de transformação sintática, que efetivamente reescreve o código. Entre as técnicas mais usuais, destacam-se a eliminação de código morto (instruções inacessíveis ou inúteis), a simplificação de redundâncias aritméticas e a movimentação estratégica de cálculos para fora de zonas de alta repetição (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

2.1.8 Geração de Código Objeto e Mapeamento de Arquitetura

~~Os fundamentos, os desafios e as estratégias que regem a etapa de geração de código objeto e o mapeamento de arquitetura aqui delineados baseiam-se nas contribuições clássicas de Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).~~

A fase de geração de código objeto (ou geração de código de máquina) representa a culminação do processo de compilação, consolidando as atividades finais do *back-end*. O propósito central desta etapa é converter a representação intermediária otimizada na linguagem nativa da plataforma de destino. Para tanto, o compilador deve assegurar não apenas a fidelidade lógica em relação ao código original, mas também o aproveitamento máximo dos recursos de *hardware* para garantir uma execução de alta performance (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Um dos obstáculos primordiais nesta fase reside na acentuada heterogeneidade das arquiteturas de hardware. Cada família de processadores, a exemplo das arquiteturas x86, ARM ou RISC-V, possui sua própria Arquitetura do Conjunto de Instruções (ISA - *Instruction Set Architecture*). Consequentemente, as operações de baixo nível e suas respectivas codificações binárias variam drasticamente entre diferentes sistemas, exigindo um mapeamento preciso para cada caso (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Para mitigar a complexidade de desenvolver um compilador do zero para cada novo processador, a engenharia de software moderna adota o princípio da *retargetabilidade* (redirecionamento). Essa modularidade é viabilizada pelo papel central da representação intermediária: enquanto o *front-end* opera de forma agnóstica em relação ao hardware, a adaptação para uma nova arquitetura exige apenas a substituição ou reconfiguração do módulo gerador de código (*back-end*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A síntese do código para uma máquina específica fundamenta-se em duas atividades interdependentes:

- **Seleção de Instruções:** Atua como um mecanismo de reconhecimento de padrões que mapeia o fluxo do código intermediário para a sequência de comandos da ISA do processador que apresente o menor custo computacional possível (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Alocação de Registradores:** Gerencia o conjunto limitado de registradores físicos da Unidade Central de Processamento (UCP). Esta tarefa é crítica, pois o compilador deve decidir estrategicamente quais valores devem permanecer nesses componentes de altíssima velocidade e quais devem ser transferidos para a memória principal (*spilling*), visando minimizar os atrasos de acesso (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

2.1.9 Engenharia de Software

Execução de Código de Máquina

~~A dinâmica operacional do código de máquina e a simbiose entre o *hardware* e o sistema operacional, discutidas a seguir, encontram seu fundamento teórico nas obras de Cooper e Torczon (2012), Nystrom (2021) e Price e Toscani (2001).~~

Ao término do processo de síntese, o que emerge é uma sucessão binária, densa e linear de operações primitivas. Diferente das abstrações hierárquicas manipuladas nas etapas iniciais da compilação, como as árvores sintáticas, o código de máquina é desprovido de estruturas de alto nível. Trata-se de uma linguagem de baixíssimo nível, constituída por instruções rudimentares que ocupam apenas alguns *bytes* e ditam ações imediatas nos circuitos: movimentação de dados entre células de memória e registradores ou a execução de operações aritméticas básicas (COOPER; TORCZON, 2012; NYSTROM, 2021).

Para que essa massa de dados binários ganhe vida, o programa deve ser transferido para a memória principal. Neste estágio, o sistema operacional, auxiliado por um *software* carregador (*loader*), prepara o ambiente de tempo de execução (*runtime*). Este processo envolve a segmentação da memória em áreas distintas, isolando o setor de código estruturado das regiões destinadas ao armazenamento de dados (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

A partir desse momento, o comando da execução passa à Unidade Central de Processamento (UCP). O "guia" dessa jornada é um registrador especializado da arquitetura, conhecido como Contador de Programa (*Program Counter* - PC) ou Ponteiro de Instrução (*Instruction Pointer* - IP), cuja função é monitorar rigorosamente o endereço da próxima instrução a ser processada (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

A execução ocorre por meio de um ciclo mecânico e ininterrupto de busca e execução. O processador percorre a fita de instruções, decodifica o código de operação (*opcode*) de cada entrada para extrair seu significado semântico e, em seguida, executa a tarefa correspondente. Nesse ecossistema de baixo nível, a complexidade dos laços de repetição e desvios condicionais das linguagens de alto nível é simplificada drasticamente. O controle do fluxo é resolvido exclusivamente através de operações de salto (*jumps* ou *branches*), que alteram o fluxo ao escrever um novo endereço de memória diretamente no Contador de Programa, desviando o "foco" da UCP de um ponto do binário para outro (COOPER; TORCZON, 2012; NYSTROM, 2021).

2.1.9.1 Gestão de Tarefas e Metodologias Ágeis

~~Para a fundamentação teórica acerca da gestão de escopo e da adoção de metodologias ágeis neste trabalho, esta subseção baseia-se principalmente nas diretrizes do *framework* Scrum detalhadas por Sabbagh (2014), bem como nos princípios de desenvolvimento iterativo e~~

~~incremental discutidos por AdaptWorks (2012). Para manter a fluidez da leitura, os conceitos de planejamento, empirismo e gerenciamento de tarefas apresentados a seguir refletem a consolidação das práticas e das ideias desses autores.~~

O gerenciamento clássico de projetos de software, frequentemente baseado no modelo em cascata (*waterfall*), assume que o desenvolvimento pode ser tratado como uma sequência previsível e linear de atividades. No entanto, a engenharia de software lida com sistemas intrinsecamente complexos e com alto grau de incerteza, onde o engessamento de requisitos em fases iniciais costuma resultar em enorme desperdício e na entrega de produtos que não atendem às reais necessidades do cliente. Em contraponto a essa visão, as metodologias ágeis assumem a imprevisibilidade natural do desenvolvimento, baseando-se no empirismo e em ciclos de *feedback* constante para promover a adaptação contínua e a entrega contínua de valor (ADAPTWORKS, 2012; SABBAGH, 2014).

Neste contexto de ambientes complexos e mutáveis, o Scrum destaca-se como um *framework* ágil, simples e focado na eficácia. Em vez de depender de cronogramas rígidos de longo prazo (como os tradicionais gráficos de Gantt), a gestão de tarefas ocorre de forma empírica e adaptativa. Todo o trabalho a ser realizado no produto é centralizado e organizado em uma lista dinâmica, ordenada e emergente, denominada *Product Backlog*. Esta estrutura substitui os extensos e estáticos documentos de requisitos tradicionais, garantindo que o escopo seja gradualmente detalhado apenas com o nível de informação estritamente necessário para o momento da execução, evitando documentações prematuras (SABBAGH, 2014).

Para viabilizar essa execução com excelência, o desenvolvimento é fatiado em ciclos curtos e de duração fixa, chamados de *Sprints*. A cada início de ciclo, as tarefas mais prioritárias são extraídas do topo do *Product Backlog* e detalhadas de forma atômica no *Sprint Backlog*. Essa divisão iterativa e atômica do trabalho permite que uma equipe multidisciplinar e auto-organizada foque exclusivamente em metas de negócio de curto prazo. Consequentemente, o projeto avança passo a passo, mitigando os riscos tecnológicos, abraçando a mudança como uma vantagem competitiva e permitindo a reavaliação contínua das prioridades do software (ADAPTWORKS, 2012; SABBAGH, 2014).



2.1.9.2 Testes Automatizados e *Test-Driven Development* (TDD)

~~Para a fundamentação da estratégia de testes e da adoção do *Test-Driven Development* (TDD), esta subseção apoia-se nos princípios de qualidade e profissionalismo discutidos por~~

~~Martin (2011), na estruturação arquitetural de testes defendida por Martin (2019) e na literatura específica de testes e *design* de software de Aniche (2012) e Silveira et al. (2012).~~

A garantia da qualidade de um software exige validação contínua de seus comportamentos. Em projetos modernos, a automação de testes reduz a dependência de verificação manual e fornece uma rede de segurança para alterações, refatorações e evolução do código. Essa prevenção contra a quebra de comportamentos já consolidados é conhecida como teste de regressão; além de validar funcionalidades, os testes automatizados funcionam como documentação executável das regras de negócio do sistema (ANICHE, 2012; SILVEIRA et al., 2012).

Nesse contexto, o TDD inverte o ciclo tradicional de implementação, exigindo que o desenvolvedor escreva um teste automatizado antes da funcionalidade. O ciclo **vermelho-verde-refatora** consiste em criar primeiro um teste que falha, escrever o código mínimo para fazê-lo passar e, por fim, refatorar a solução mantendo o comportamento validado (ANICHE, 2012; MARTIN, 2011).

Mais do que uma técnica de verificação, o TDD também influencia o *design* de software. Ao escrever o teste primeiro, o desenvolvedor precisa pensar na interface pública, nas dependências e na responsabilidade de cada componente; dificuldades excessivas na escrita dos testes podem indicar acoplamento elevado, baixa coesão ou problemas de arquitetura (MARTIN, 2019; ANICHE, 2012).



2.1.9.3 Arquitetura Limpa e Separação de Responsabilidades

~~Para a fundamentação das decisões arquiteturais e dos padrões de projeto adotados neste trabalho, esta subseção baseia-se nos preceitos de Arquitetura Limpa propostos por Martin (2019) e nas diretrizes de *design* de software estruturadas por Silveira et al. (2012).~~

O projeto arquitetural de um sistema deve separar responsabilidades em camadas com papéis independentes e bem definidos. A interface de usuário no *front-end* concentra-se na experiência de uso e na integração visual, enquanto regras de negócio, persistência e serviços de apoio devem permanecer desacoplados de detalhes periféricos de entrega. Essa separação reduz dependências indevidas e facilita testes, manutenção e evolução do software (MARTIN, 2019; SILVEIRA et al., 2012).

No contexto da plataforma, a separação entre elementos de interface, regras de customização da linguagem, execução do código e persistência de dados contribui para que cada parte possa

evoluir sem comprometer as demais. Essa organização também favorece o uso de padrões de projeto e abstrações voltadas à redução de acoplamento entre os módulos (MARTIN, 2019).

2.1.9.4 Qualidade de Código e *Clean Code*

~~Para o detalhamento conceitual e a definição das práticas de *Clean Code* (Código Limpo) descritas nesta subseção, este trabalho fundamenta-se nas consolidadas diretrizes de Martin (2009) e na literatura de apoio sobre metodologias ágeis abordada por AdaptWorks (2012). Para garantir a fluidez da leitura, toda a argumentação estrutural, as definições de qualidade e as regras de formatação apresentadas a seguir derivam da visão e das heurísticas desses autores.~~

A construção de um software **ágil e sustentável** esbarra invariavelmente na qualidade de sua base de código. O código-fonte é a linguagem formal na qual os requisitos de um sistema são expressos e detalhados de modo que a máquina possa executá-los. No entanto, sob a pressão de prazos, é comum que as equipes produzam códigos desorganizados, resultando em uma degradação estrutural contínua que, a longo prazo, faz a produtividade despencar, podendo até mesmo inviabilizar a evolução do projeto (MARTIN, 2009; ADAPTWORKS, 2012).

Para combater essa entropia, a adoção das práticas de Código Limpo torna-se essencial. A escrita de um código limpo exige disciplina, o uso consciente de padrões e uma sensibilidade aguçada para reconhecer a desorganização e aplicar refatorações. Um código limpo deve ser elegante, simples, direto e, sobretudo, legível para os seres humanos. O profissionalismo no desenvolvimento de software pressupõe que o código não deve apenas funcionar, mas deve ser fácil de entender, alterar e estender por outros desenvolvedores (MARTIN, 2009).

Dentre as diretrizes estruturais básicas e inegociáveis para a manutenção dessa clareza na base de código, destacam-se (MARTIN, 2009):

- **Nomes Significativos:** A nomenclatura de variáveis, funções, parâmetros e classes deve revelar claramente a sua intenção e responder ao porquê de sua existência e ao que o componente faz. Deve-se evitar o uso de siglas obscuras, codificações desnecessárias ou trocadilhos que exijam mapeamentos mentais, facilitando assim a leitura fluida e a busca no código (MARTIN, 2009).
- **Funções Pequenas e Focadas:** Uma função deve ser extremamente concisa e projetada para fazer estritamente uma única coisa, mantendo um único nível de abstração. Funções extensas que realizam múltiplas tarefas, agrupam várias seções lógicas ou que misturam

tratamento de erros com a lógica de negócio violam a organização e dificultam o entendimento e a testabilidade (MARTIN, 2009).

- **Uso Consciente de Comentários:** O uso de comentários costuma compensar a incapacidade do programador de se expressar claramente através do próprio código. Eles não devem ser utilizados para encobrir ou justificar um código ruim ou confuso. A energia da equipe deve ser direcionada para tornar o código tão claro e autoexplicativo que elimine a necessidade de anotações redundantes (MARTIN, 2009).
- **Eliminação de Duplicidade e Refatoração contínua:** A repetição de código é o grande inimigo de um sistema bem desenvolvido, pois representa trabalho, risco e complexidade desnecessária. Deve-se melhorar o design do código de forma contínua, aplicando refatorações que tornem o sistema mais enxuto e compreensível, sem alterar o seu comportamento externo (MARTIN, 2009; ADAPTWORKS, 2012).

2.1.9.5 O Framework Next.js e sua Arquitetura

~~Para a fundamentação teórica acerca dos conceitos arquiteturais, da evolução do sistema de roteamento e das estratégias de renderização do *framework* Next.js, os parágrafos a seguir foram baseados nas diretrizes e documentações técnicas apresentadas por Ziemann (2025), Lai et al. (2025), MakerKit (2026) e Vercel (2024).~~

O Next.js é um *framework* React voltado para a construção de aplicações web *full-stack*, que facilita o desenvolvimento ao configurar internamente ferramentas de baixo nível, como empacotadores e compiladores, permitindo que o foco seja direcionado exclusivamente à construção do produto. A evolução de sua arquitetura ficou marcada pela transição do modelo tradicional de rotas, conhecido como *Pages Router*, para o *App Router*. Esse novo modelo adota uma arquitetura avançada centrada em *React Server Components*, aprimorando o processo de busca de dados e permitindo o uso nativo de *layouts* aninhados por meio de uma estrutura rigorosa de arquivos, utilizando convenções como `layout.tsx` e `loading.tsx` (ZIEMANN, 2025; VERCEL, 2024).

A adoção de componentes de servidor transforma a forma de renderização, pois permite que o código seja executado exclusivamente no servidor e nunca seja enviado ao cliente, o que elimina a sobrecarga de arquivos JavaScript redundantes ao navegador. Integrado a essa estrutura arquitetural, o *framework* disponibiliza o *streaming*, uma técnica muitas vezes apoiada pelo

React Suspense, que permite enviar partes da interface gráfica progressivamente para o cliente à medida que os dados ficam prontos, reduzindo o tempo de espera inicial percebido pelo usuário (LAI et al., 2025; VERCEL, 2024).

Outra característica de destaque, consolidada nas versões mais recentes, é o uso de *Server Actions*. Tais ações consistem em funções assíncronas executadas diretamente no servidor que podem ser invocadas a partir de componentes interativos no cliente. Essa abordagem simplifica mutações de dados complexas, como submissões de formulários e interações com o banco de dados, eliminando a necessidade de criar arquivos manuais para *endpoints* de API (MAKERKIT, 2026; VERCEL, 2024).

Por fim, para a otimização em ambiente de produção, a ferramenta não apenas suporta múltiplas estratégias de renderização, como a estática e a dinâmica, mas também possibilita a escolha do ambiente de execução (*runtime*) mais adequado para cada rota. O desenvolvedor pode optar pelo *Node.js Runtime* padrão, que possui acesso completo a todas as APIs e ao ecossistema do Node.js, ou pelo *Edge Runtime*, que opera com base em APIs da Web e é projetado para latência mínima e inicialização rápida ao lidar com lógicas de resposta simples (MAKERKIT, 2026; VERCEL, 2024).

2.1.9.6 Design de Experiência do Usuário (UX) e Interface do Usuário (UI)

~~Para a fundamentação teórica acerca dos conceitos, diferenças e aplicabilidades do *User Experience* (UX) e *User Interface* (UI), os parágrafos a seguir foram baseados em Seyffert (2026), Yablonski (2020), Lima (2024) e Raposo (2012).~~

O **design** de produtos digitais modernos compreende duas disciplinas fundamentais e complementares: a Experiência do Usuário (UX) e a Interface do Usuário (UI). A UX refere-se à percepção, às emoções e às respostas de um indivíduo ao interagir com um sistema ao longo de toda a sua jornada. Essa área vai muito além da usabilidade, englobando a compreensão profunda do comportamento, das necessidades e das frustrações do público-alvo. Para garantir uma boa experiência, a organização estrutural é essencial, sendo a Arquitetura da Informação a especialidade responsável por moldar, gerenciar e dispor os conteúdos de modo a tornar a navegação fluida, acessível e agradável (SEYFFERT, 2026; LIMA, 2024; RAPOSO, 2012).

Por outro lado, a UI diz respeito à camada visual e interativa com a qual o usuário interage diretamente. Isso inclui a definição rigorosa de layouts, botões, tipografia, cores e componentes visuais. Mais do que atuar apenas na estética, uma interface bem projetada deve garantir clareza

e consistência de uso. Uma organização confusa no aspecto visual pode comprometer toda a experiência global e levar ao abandono do sistema, independentemente de quão robusta seja a sua funcionalidade em *back-end* (SEYFFERT, 2026; LIMA, 2024).

A relação entre essas duas áreas é de forte complementaridade, na qual a UX estrutura a lógica e a solução do problema, enquanto a UI materializa essas soluções em uma interface tangível e eficiente. O desenvolvimento adequado em engenharia de software dita que a estruturação de UX anteceda a de UI; ou seja, define-se primeiramente quais problemas serão resolvidos e como os fluxos de navegação funcionarão, para somente então aplicar o design visual (SEYFFERT, 2026; RAPOSO, 2012). 

Essa intersecção técnica também é fortemente influenciada por aspectos cognitivos e leis da psicologia. O chamado “efeito estética-usabilidade”, por exemplo, demonstra que os usuários tendem a perceber um design esteticamente agradável, isto é, uma boa UI, como sendo inerentemente mais utilizável e eficiente na resolução de suas tarefas. Para avaliar e garantir a eficácia dessas interfaces, métodos de inspeção como a avaliação heurística são comumente aplicados. Trata-se de uma análise guiada por heurísticas (princípios gerais de bom design) para certificar itens como a visibilidade do estado do sistema e a prevenção de erros, maximizando a usabilidade final do artefato (YABLONSKI, 2020; RAPOSO, 2012).

2.2 Softwares Educativos e Tecnologias de Apoio à Aprendizagem

~~Para a fundamentação conceitual acerca do uso de tecnologias no ambiente escolar abordada nesta subseção, este trabalho apoia-se nas perspectivas sobre plataformas educativas delineadas por Benedetti (2025) e nas diretrizes de inovação em tecnologia educacional da Mind Makers (2025). Com o intuito de garantir a fluidez da leitura, os conceitos de mediação tecnológica, personalização do ensino e sistemas de apoio apresentados a seguir refletem a consolidação das ideias desses autores.~~

Uma plataforma educativa é um ambiente virtual desenvolvido para apoiar processos de ensino e aprendizagem, oferecendo recursos, ferramentas e conteúdos que facilitam a interação entre alunos, professores e instituições de ensino. Nesse cenário de transformação, a escolha da ferramenta certa não é apenas uma questão tecnológica, mas um fator determinante para o engajamento, a motivação e o sucesso educacional (BENEDETTI, 2025; Mind Makers, 2025).

Contudo, para aproveitar o potencial das novas tecnologias, a instituição de ensino

precisa ir além da simples adoção de ferramentas digitais. É essencial entender o propósito pedagógico por trás de cada inovação, garantindo que a tecnologia esteja sempre a serviço da aprendizagem, e não o contrário. Afinal, a tecnologia educacional atua como um meio que exige professores preparados para usar recursos digitais de forma criativa e estratégica, conectando os alunos a um aprendizado relevante, dinâmico e significativo (Mind Makers, 2025).

Quando projetada com esse foco pedagógico, uma plataforma educativa bem estruturada permite a personalização do aprendizado, ajustando-se ao ritmo e ao estilo de cada aluno, o que maximiza seu potencial e promove uma assimilação muito mais eficaz do conteúdo. Essa abordagem valoriza o tempo de cada estudante e amplia as suas capacidades ao longo de toda a jornada educacional. Ademais, a integração de dinâmicas interativas nas ferramentas de ensino, como a gamificação e a aprendizagem baseada em desafios, transforma a educação em um processo envolvente de recompensas. Esses sistemas lúdicos ajudam a despertar o interesse contínuo dos alunos e a desenvolver habilidades socioemocionais indispensáveis, como persistência e colaboração (BENEDETTI, 2025; Mind Makers, 2025).



2.3 Trabalhos Relacionados

Esta seção tem como finalidade situar a plataforma desenvolvida neste trabalho no conjunto de soluções tecnológicas voltadas ao ensino introdutório de programação. Embora o objetivo do projeto seja apoiar a aprendizagem de programação, e não o ensino de construção de compiladores, sua própria natureza, a de uma IDE acoplada a um interpretador de gramática configurável, posiciona a proposta na fronteira entre dois universos de ferramentas: de um lado, os ambientes voltados à iniciação em programação; de outro, os projetos didáticos de construção de compiladores e de analisadores léxicos e sintáticos, cujas decisões técnicas e pedagógicas são pertinentes por dialogarem diretamente com o núcleo de tradução que sustenta esta plataforma. Por essa razão, são examinadas cinco iniciativas representativas do estado da arte abrangendo ambos os grupos, observando, em cada uma delas, as escolhas de interface, os recursos pedagógicos e as estratégias de interação com o estudante. Ao final, apresenta-se uma análise comparativa que evidencia as lacunas restantes e o diferencial da abordagem proposta, que se situa justamente entre esses dois grupos.

2.3.1 Portugal Studio e Portugal Webstudio

~~Os elementos descritivos e funcionais do Portugal Studio e de sua versão executada em navegador, apresentados nesta subseção, apoiam-se nos estudos de GOMES (2020) e MARTINS (2025), bem como na documentação institucional disponibilizada pela UNIVALI (2026).~~

Criado pelo Laboratório de Inovação Tecnológica na Educação (LITE) da Universidade do Vale do Itajaí, o Portugal Studio é distribuído como um ambiente de desenvolvimento integrado de código aberto cujo público-alvo são estudantes em contato inicial com a lógica de programação. A plataforma apoia-se em uma linguagem inspirada nas construções sintáticas de C e PHP, porém com palavras-chave totalmente traduzidas para o português brasileiro, o que evita que o aprendiz precise lidar com dois idiomas ao mesmo tempo durante o estudo das estruturas básicas de controle de fluxo e de dados.

Do ponto de vista da experiência do usuário, a ferramenta investe em recursos didáticos de visualização interna do programa: depurador com execução passo a passo, inspetor de variáveis sincronizado com a linha em execução e uma árvore que representa a estrutura sintática do código produzido. A migração posterior para o Portugal Webstudio reforça a preocupação com acessibilidade tecnológica, pois leva toda a edição, compilação e execução para o navegador, eliminando a barreira da instalação local em laboratórios escolares com configurações diferentes. Soma-se a isso o suporte nativo a bibliotecas gráficas e sonoras, recurso que permite criar projetos lúdicos, como pequenos jogos, e tende a elevar o engajamento dos estudantes durante o semestre letivo.

2.3.2 Laila: Gerador de Analisadores Léxicos e Sintáticos Online

~~A caracterização da arquitetura e os resultados pedagógicos da plataforma Laila, discutidos a seguir, baseiam-se na pesquisa conduzida por Silva (2021).~~

A motivação central da proposta de Laila é remover o atrito de configuração que historicamente acompanha a disciplina de Compiladores. A instalação coordenada de geradores como FLEX e Bison junto a uma cadeia compatível do GCC frequentemente consome parte significativa da carga horária de laboratório, sobretudo em ambientes operacionais que não derivam do Linux. Em resposta a esse obstáculo, a ferramenta oferece um IDE inteiramente executado no navegador, no qual o estudante especifica gramáticas, redige analisadores léxicos e sintáticos e executa os experimentos sem dependências locais.

A arquitetura adotada organiza-se em uma camada de *front-end* construída em JavaScript com o *framework* VueJS, complementada pela biblioteca CodeMirror, responsável pelo destaque de sintaxe e pela sinalização inline de inconsistências. A camada de *back-end* corresponde a uma API REST em Python, responsável por coordenar a execução de FLEX e Bison no servidor, capturar comportamentos anômalos como laços infinitos ou erros de compilação e devolver os respectivos diagnósticos ao cliente. Estudos de caso conduzidos junto a turmas de Ciência da Computação relataram ganhos perceptíveis de engajamento e de compreensão prática dos algoritmos de análise, atribuídos ao ciclo curto entre edição, submissão e retorno do resultado.

2.3.3 Quorum: Linguagem de Programação Baseada em Evidências

~~Os princípios de projeto, os resultados experimentais e os recursos de ambiente da linguagem Quorum, abordados nesta subseção, fundamentam-se nas investigações de Stefik e Siebert (2013), no relato de Ladner (2019) e na documentação oficial do projeto QUORUM (2025).~~

O Quorum nasce como resposta a uma preocupação específica: tornar a programação nativamente acessível a estudantes com deficiência visual, cuja interação com a linguagem ocorre principalmente por meio de leitores de tela. A partir desse compromisso inicial, o projeto amadureceu para uma linguagem de propósito geral cuja característica distintiva é adotar decisões de sintaxe e de semântica baseadas em estudos empíricos, em oposição às escolhas tradicionalmente herdadas por imitação ou por familiaridade histórica.

Os testes conduzidos pela equipe responsável incluíram comparações entre linguagens consolidadas no mercado e uma linguagem de controle cujas palavras-chave foram geradas aleatoriamente, recurso metodológico que permitiu isolar o efeito da forma sintática sobre o desempenho de novatos. As evidências reunidas indicaram que a substituição de símbolos como chaves, colchetes e pontos e vírgulas por palavras de vocabulário comum reduz de modo consistente as taxas de erro sintático. Esse resultado aparece em construções como `repeat 10 times`, cuja leitura se aproxima de uma instrução em linguagem natural. Para sustentar essa proposta no uso cotidiano, o ecossistema disponibiliza o Quorum Studio, IDE acessível desde sua concepção, capaz de operar tanto em modo *offline*, com geração de *bytecode* para a JVM, quanto em modo executável no navegador, com suporte a aplicações que vão de exercícios introdutórios a projetos de áudio, gráficos e robótica.

2.3.4 **chibicc: Uma Abordagem Incremental para o Ensino de Compiladores**

~~A caracterização do compilador *chibicc* e de sua estratégia educacional baseia-se no projeto e na documentação técnica disponibilizados por Ueyama (2020).~~

O *chibicc* é um pequeno compilador para a linguagem C, projetado para servir como referência didática no ensino de construção de compiladores e de programação de sistemas. Apesar do enfoque pedagógico, o projeto cobre uma parte expressiva da especificação C11, sendo capaz de compilar, sem adaptações, projetos de complexidade considerável, como o sistema de controle de versão Git e o gerenciador de banco de dados SQLite.

O traço pedagógico mais relevante da ferramenta está em sua organização incremental. Em vez de expor o aprendiz, desde o princípio, a uma arquitetura completa com múltiplos passes e estruturas internas elaboradas, o projeto propõe uma trajetória gradual em que cada etapa adiciona uma única capacidade gramatical ou semântica ao compilador. O ponto de partida é um tradutor mínimo que reconhece apenas um número como entrada válida; sobre essa base inicial, são integrados, em iterações sucessivas, suporte a expressões, declarações, controle de fluxo, ponteiros, estruturas e demais elementos do C11. Essa progressão favorece a observação direta da relação entre as escolhas de implementação e o comportamento final do compilador, oferecendo uma porta de entrada mais acessível ao estudo de baixo nível.

2.3.5 **Beecrowd: Prática de Algoritmos e Avaliação Automatizada**

~~A descrição da plataforma Beecrowd, de seus mecanismos de avaliação automatizada e de seus efeitos no aprendizado de algoritmos, apresentada nesta subseção, fundamenta-se no estudo de caso de Ferreira, Silva e Ribeiro (2025), no artigo de Garcia (2024) e no material complementar de MÃO NO CÓDIGO (2022).~~

Antes denominada URI Online Judge, a Beecrowd consolidou-se como um juiz online amplamente adotado no ambiente acadêmico brasileiro e em treinamentos voltados às maratonas de programação. Seu acervo contempla milhares de problemas categorizados por temas e por níveis de dificuldade, abrangendo desde estruturas básicas de seleção e repetição até tópicos avançados como grafos e programação dinâmica, e admite submissões em diversas linguagens consolidadas no mercado, incluindo C++, Python, Java e JavaScript.

A dinâmica de uso da plataforma articula-se em torno de um ciclo curto de submissão e *feedback*: o estudante lê o enunciado, redige uma solução e a envia ao sistema, que executa o

código contra um conjunto fechado de casos de teste e retorna, em poucos segundos, um veredito como “Accepted”, “Wrong Answer” ou “Presentation Error”. Esse retorno imediato, combinado a elementos de gamificação como *rankings* públicos, fóruns e contagem de submissões aceitas, sustenta um regime de prática deliberada que tende a manter o estudante em posição ativa e a estimular a exploração contínua de novos problemas.

2.3.6 Análise Crítica e o Diferencial da Solução Proposta

~~A análise crítica desenvolvida nesta subseção organiza-se em torno das características discutidas anteriormente, tomando como apoio os trabalhos de Silva (2021), Stefik e Siebert (2013), Ueyama (2020), Ferreira, Silva e Ribeiro (2025) e a documentação institucional da UNIVALI (2026).~~ A Tabela 2.1 sintetiza as principais características das ferramentas examinadas frente à plataforma proposta, evidenciando, em especial, a personalização da sintaxe pelo próprio estudante como traço distintivo da abordagem.

Tabela 2.1 – Comparativo entre as ferramentas analisadas e a plataforma proposta.

Ferramenta	Público-alvo	Web	Gramática	Personalização da sintaxe pelo usuário	Avaliação automatizada
Portugol Studio	Iniciante	Sim	Fixa (PT-BR)	Não	Não
Quorum	Iniciante	Sim	Fixa (baseada em evidências)	Não	Não
Beecrowd	Iniciante a avançado	Sim	Fixa (linguagens de mercado)	Não	Sim
Laila	Avançado	Sim	Definida pelo usuário (gramáticas formais)	Não (foco em compiladores)	Não
chibicc	Avançado	Não	Fixa (C11)	Não	Não
Plataforma proposta	Iniciante	Sim	Configurável pelo usuário	Sim: palavras-chave, delimitadores de bloco, operadores em forma de palavra, literais booleanos e terminador de instrução	Sim

Fonte: elaborado pelo autor.

Quando observadas em conjunto, as ferramentas analisadas organizam-se em dois grandes grupos cujas limitações afetam diretamente o público iniciante. O primeiro grupo reúne soluções dedicadas à prática algorítmica e à formação inicial em programação, do qual fazem parte o Portugol Studio, o Quorum e a Beecrowd. Embora apresentem avanços relevantes, seja por meio da tradução das palavras-chave para o português, da adoção de uma sintaxe baseada em

evidências ou da prática contínua sustentada por avaliação automatizada, todas operam sob uma gramática fixa, definida previamente pelos seus responsáveis. O estudante permanece, portanto, na condição de quem se adapta à linguagem, e não de quem participa de sua modelagem; pequenas diferenças entre o vocabulário esperado e o vocabulário internalizado podem invalidar soluções logicamente corretas e reintroduzir, ainda que de forma reduzida, a barreira sintática que se buscava remover.

O segundo grupo reúne plataformas e projetos voltados ao ensino de compiladores propriamente dito, exemplificados pela Laila e pelo *chibicc*. Ambos oferecem contribuições expressivas em seus nichos, sobretudo na eliminação do atrito de configuração ou na apresentação incremental da arquitetura de um compilador. Contudo, o público pressuposto é, em grande medida, o de estudantes em estágio avançado de cursos de Computação, com domínio prévio de linguagens estritas como C11 e de geradores como FLEX e Bison, o que afasta esses recursos do estudante que ainda enfrenta a barreira inicial de entrada no pensamento computacional.

A plataforma proposta situa-se entre esses dois grupos. Em vez de oferecer uma gramática previamente fixada ou de exigir a configuração local de cadeias de ferramentas, este trabalho disponibiliza uma IDE web integrada a um interpretador dinâmico cuja função principal é deslocar o controle estrutural da linguagem para o próprio estudante. Por meio da interface, é possível redefinir palavras-chave, delimitadores de bloco, operadores expressos por palavras, literais booleanos e regras de finalização de instrução, sem que o aprendiz precise lidar com a complexidade interna das fases de análise léxica, sintática e semântica.

Do ponto de vista da experiência do usuário, a personalização oferecida pela plataforma transforma o ato de programar em uma atividade na qual a linguagem se ajusta ao repertório do aprendiz, e não o contrário. Esse arranjo cria um laboratório seguro de experimentação algorítmica, no qual a frustração associada a erros de digitação ou de memorização de palavras reservadas é substituída pela investigação consciente das estruturas de controle e de dados. Forma-se, assim, uma ponte pedagógica entre a primeira aproximação ao raciocínio computacional e o futuro contato do estudante com as linguagens estritas e com os juízes automatizados que predominam no mercado e nas etapas mais avançadas da formação.

3 PROPOSTA

A finalidade deste capítulo é delimitar o que se pretende construir ao longo das duas fases do Trabalho de Conclusão de Curso e, em seguida, apresentar a cadeia temporal de atividades prevista na pré-proposta aprovada. A descrição parte da plataforma vista como produto educacional, atravessa as macroatividades técnicas necessárias à sua materialização e culmina na exposição do cronograma e do regime de trabalho em dupla adotado.

O artefato que orienta este trabalho não é apenas um compilador ou um editor de código, mas sim uma **plataforma educacional web** cuja unidade de valor é a possibilidade de o estudante personalizar a própria linguagem que utilizará para aprender programação. **Em termos concretos, oferece-se ao usuário a possibilidade de redefinir o vocabulário das palavras-chave, escolher os delimitadores de bloco que melhor representem sua noção de escopo, substituir operadores lógicos e relacionais por palavras de uso natural, reescrever os literais booleanos no idioma de sua preferência e ajustar as regras de finalização de instrução.** Cada uma dessas dimensões responde, em alguma medida, a uma fonte conhecida de atrito sintático no contato inicial com linguagens consolidadas no mercado, sendo a sua reunião sob uma mesma interface o que sustenta o caráter inovador da proposta.

Conforme definido na pré-proposta, a metodologia foi organizada em seis macroetapas complementares. Essas etapas não devem ser entendidas como blocos completamente isolados, pois o desenvolvimento de uma plataforma interativa exige revisões sucessivas entre interface, interpretador e experiência de uso. Ainda assim, elas delimitam com clareza o percurso técnico e acadêmico do projeto:

- **Revisão bibliográfica:** levantamento das bases conceituais sobre interpretadores, **compiladores** e ferramentas educacionais relacionadas, com o objetivo de sustentar tanto as decisões de projeto quanto a posterior análise comparativa;
- **Definição de requisitos e arquitetura:** consolidação dos elementos configuráveis da linguagem, da modelagem das camadas da plataforma e dos contratos de comunicação entre o núcleo do interpretador e a IDE;
- **Desenvolvimento da plataforma:** implementação da aplicação web, contemplando os fluxos de edição, navegação, personalização, autenticação e uso educacional da IDE;

- **Implementação da análise dinâmica léxica e sintática:** integração entre o código escrito na IDE e as fases de reconhecimento de *tokens* e validação sintática com suporte às regras configuradas pelo usuário;
- **Geração e interpretação do código intermediário:** disponibilização, no ambiente da IDE, da execução do código intermediário e do retorno de avisos, erros e resultados ao usuário;
- **Integração e testes entre plataforma e interpretador:** validação dos fluxos principais, com atenção à consistência da experiência, à acessibilidade, à responsividade e à preparação para os estudos com usuários.

A distribuição temporal dessas atividades ao longo dos doze meses que compreendem o TCC I e o TCC II é apresentada na Tabela 3.1, conforme o cronograma registrado na pré-proposta. Cada coluna numerada corresponde a um mês corrido das duas etapas e cada célula marcada com “X” sinaliza a previsão de execução da respectiva atividade no período indicado.

Tabela 3.1 – Cronograma de desenvolvimento do trabalho de conclusão de curso.

Atividade	TCC I						TCC II					
	01	02	03	04	05	06	07	08	09	10	11	12
Revisão Bibliográfica	X	X	X	X								
Escrita da Introdução e Revisão de Literatura		X	X	X								
Predefinição de escopo e Especificação dos comandos dinâmicos			X	X								
Escrita da Metodologia			X		X	X						
Arquitetura e Modelagem da Plataforma			X	X								
Customização dinâmica de tokens e sintaxe				X	X							
Execução da Análise Léxica e Sintática com o código da IDE					X	X						
Execução do código intermediário no Terminal da IDE com <i>feedback</i> (avisos e erros)						X	X	X				
Finalização de fluxos, acessibilidade e responsividade						X	X	X				
Testes, Validação com Usuários e Coleta de <i>Feedback</i>									X	X	X	
Escrita do TCC (Resultados, Discussão e Conclusão)									X	X	X	X

Fonte: elaborado pelo autor.

Convém retomar, neste ponto, o regime de trabalho em dupla já apresentado na introdução (Capítulo 1), dada sua influência direta sobre o escopo do projeto e sobre os capítulos subsequentes.

A pré-proposta aprovada para este Trabalho de Conclusão de Curso prevê a **participação conjunta em todas as etapas** do desenvolvimento, mas com funções complementares. Em respeito a esse compromisso, as decisões de arquitetura, os ciclos de revisão de código e as etapas de integração foram conduzidos de modo colaborativo pelos dois integrantes da dupla. Cabe ressaltar que, por se tratar de uma equipe reduzida, composta por apenas dois integrantes, os papéis formais e as cerimônias completas previstos no *framework* Scrum não foram instanciados; adotaram-se, de forma pragmática, apenas as práticas essenciais de desenvolvimento iterativo e incremental discutidas no referencial teórico, como a organização do trabalho em ciclos curtos e a manutenção de uma lista priorizada de tarefas.

A partir dessa **base comum**, a divisão complementar de eixos de aprofundamento manifesta-se nos capítulos de redação acadêmica e nas evoluções incrementais subsequentes. No presente documento, o autor concentra a investigação na plataforma e na customização de comandos: arquitetura da aplicação web, concepção do assistente de personalização, experiência de uso da IDE, execução do código no terminal integrado e procedimentos de coleta de *feedback* junto aos estudantes. O detalhamento aprofundado da evolução do núcleo do interpretador, incluindo a geração de código intermediário e sua integração técnica ao restante da plataforma, é desenvolvido em paralelo pelo estudante Victor de Souza Vilela da Silva. Essa divisão de eixos não fragmenta o produto: o que se distribui é a profundidade analítica de cada autor sobre partes específicas do mesmo artefato, conduzido pelos dois em todas as etapas. Tal arranjo justifica-se pela amplitude do escopo, que, individualmente, tenderia a resultar em um interpretador simplificado ou em uma plataforma incompleta, restrita a um terminal.

Concluídas as etapas cobertas por este documento, dedicadas à fundamentação teórica, à modelagem da plataforma, à customização dinâmica de *tokens* e sintaxe, à execução das análises léxica e sintática a partir da IDE e à execução do código intermediário no terminal integrado, a fase subsequente direcionará seus esforços à **condução dos estudos de caso, à coleta de evidências empíricas em sala de aula e à análise comparativa entre a plataforma desenvolvida e as ferramentas discutidas no referencial teórico**. O próximo capítulo, dedicado à modelagem, descreve o estado em que o sistema se encontra ao término do TCC I.

4 METODOLOGIA

Este capítulo apresenta a abordagem metodológica adotada para o desenvolvimento da plataforma web proposta neste trabalho, detalhando a arquitetura do Ambiente de Desenvolvimento Integrado (IDE), as estratégias empregadas para a customização dinâmica de `tokens` e sintaxe, os mecanismos de integração com o núcleo do interpretador e os fluxos de execução, avaliação e *feedback* ao usuário. A condução do trabalho seguiu rigorosamente as etapas previstas no cronograma do projeto, contemplando, nesta primeira fase do Trabalho de Conclusão de Curso, a revisão bibliográfica, a especificação do escopo dos comandos dinâmicos, a modelagem da arquitetura da plataforma, a implementação da customização interativa da linguagem, a execução das análises léxica e sintática a partir do código produzido na IDE, a apresentação dos resultados intermediários no terminal embutido e a finalização dos fluxos principais, com atenção à `acessibilidade` e à responsividade.

O desenvolvimento foi conduzido de forma incremental e iterativa, em conformidade com os princípios do desenvolvimento ágil discutidos por AdaptWorks (2012) e Sabbagh (2014). Em razão da elevada quantidade de telas, componentes interativos e estados compartilhados característicos de uma IDE web, cada fluxo da plataforma, desde a autenticação do usuário até a submissão de exercícios, foi tratado como uma unidade autônoma de entrega, modelada, implementada e validada antes da agregação à etapa seguinte. Essa abordagem foi sustentada por testes automatizados unitários e de integração escritos com o *framework* Vitest, conforme as práticas de Desenvolvimento Guiado por Testes consolidadas por Aniche (2012) e Martin (2011), permitindo a verificação contínua de regressões diante das frequentes mudanças impostas pela natureza altamente interativa da plataforma.



4.1 Modelagem da Arquitetura da Plataforma

A organização da plataforma foi concebida sob a ótica da Separação de Responsabilidades defendida por Martin (2019), dividindo o projeto em três camadas independentes e bem delimitadas: o pacote `compiler`, responsável pelo núcleo do interpretador; o pacote `ide`, responsável pela camada de apresentação e por toda a interação com o usuário; e o serviço de *back-end*, responsável pela persistência das entidades de domínio do Ambiente Virtual de Aprendizagem. Os pacotes `compiler` e `ide` são versionados em um único repositório no formato *monorepo*, viabilizando o consumo direto do compilador como dependência local da

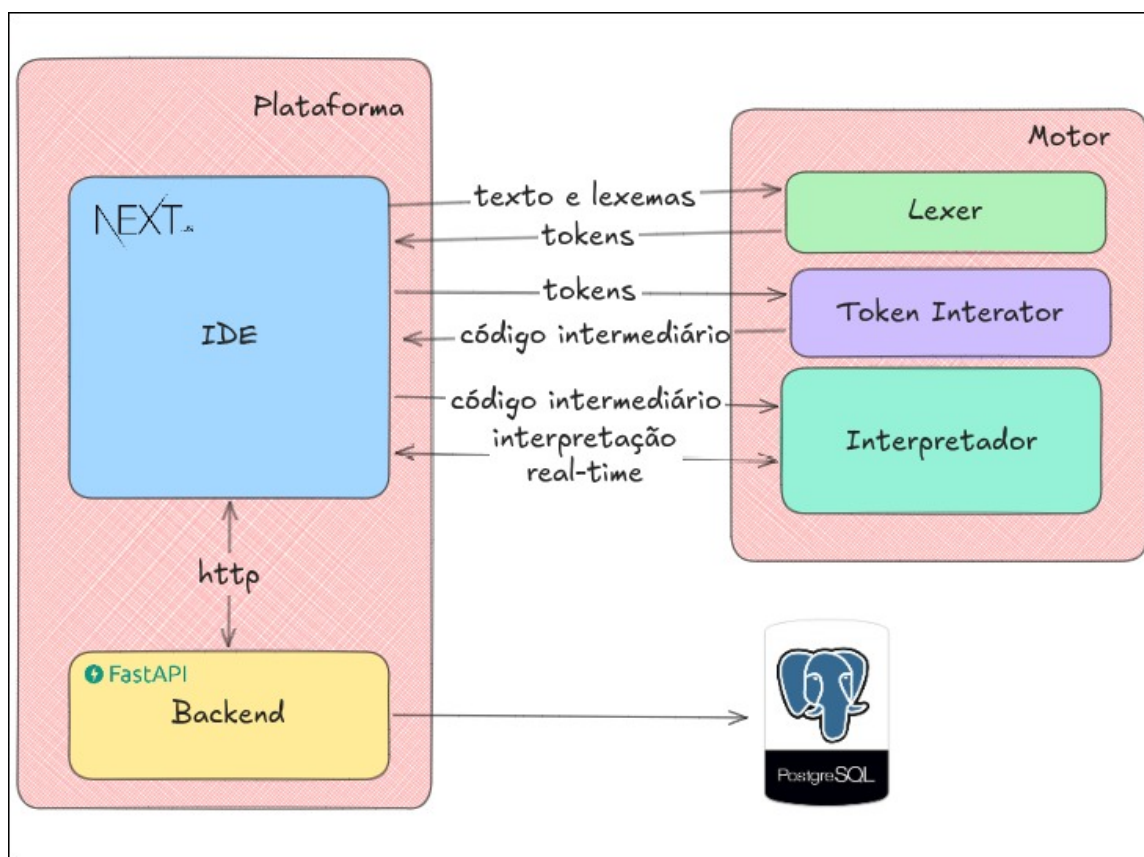
IDE, sem qualquer empacotamento intermediário e sem comunicação remota entre as duas camadas. Essa decisão arquitetural reduz drasticamente a latência da execução de programas no navegador e simplifica a propagação de mudanças no compilador para a interface.

O *front-end* foi implementado com o *framework* Next.js, em sua arquitetura de *Pages Router*, sobre a biblioteca React 19, utilizando TypeScript em todo o código-fonte para garantir a verificação estática de tipos discutida por Martin (2009). A estilização foi conduzida com Tailwind CSS em sua quarta versão, complementada por componentes acessíveis das bibliotecas *shadcn/ui* e *Radix UI*, escolhidos por implementarem as recomendações da *Web Accessibility Initiative* (WAI-ARIA) por padrão. O gerenciamento de estado assíncrono foi delegado à biblioteca *TanStack React Query*, responsável pela orquestração de cache, sincronização e revalidação dos dados consumidos das rotas HTTP, enquanto a validação de formulários foi modelada com a biblioteca *Zod*, integrada ao *React Hook Form*.

A camada de *back-end* foi implementada em Python sobre o *framework* FastAPI, escolhido pelas características de alto desempenho, validação automática via *Pydantic* e geração automatizada de documentação OpenAPI discutidas por FASTAPI (2026). A persistência foi sustentada pelo ORM SQLAlchemy assíncrono, com versionamento de *schema* via Alembic, isolando o modelo de dados das regras de negócio em conformidade com os princípios de Arquitetura Limpa apresentados por Martin (2019). Essa separação garante que o banco de dados seja tratado **estritamente como um detalhe de infraestrutura**, permitindo que o núcleo de compilação e a interface web permaneçam agnósticos em relação à camada de armazenamento.

O fluxo geral de comunicação entre as camadas, ilustrado pela Figura 4.1, foi estruturado de modo que a execução do código submetido pelo usuário ocorra integralmente nos limites do servidor Next.js. A IDE invoca rotas HTTP locais sob o caminho `/api`, que importam diretamente as classes `Lexer`, `TokenIterator` e `Interpreter` do pacote `compiler`. As requisições destinadas à persistência são, por sua vez, delegadas ao serviço FastAPI por meio do cliente HTTP *axios*. Essa segregação confere à plataforma a flexibilidade de executar programas mesmo na ausência de conexão com o serviço de persistência, mantendo plenamente funcionais os recursos de edição, análise léxica, geração de código intermediário e interpretação.

Figura 4.1 – Arquitetura geral da plataforma e fluxos de comunicação entre as camadas.



Fonte: elaborado pelo autor.

A organização interna do código da IDE seguiu a topologia clássica de aplicações Next.js, com diretórios dedicados para páginas (`pages`), componentes reutilizáveis (`components`), `contexts` de estado global (`contexts`), `hooks` personalizados (`hooks`), entidades de domínio (`entities`), bibliotecas de apoio (`lib`) e composições de tela (`views`). Essa segmentação materializa o princípio da coesão funcional discutido por Silveira et al. (2012), isolando responsabilidades específicas em módulos compactos e favorecendo tanto a legibilidade quanto a testabilidade do sistema.

4.2 Interpretador

O interpretador constitui o núcleo de tradução comum a este projeto e foi desenvolvido de forma colaborativa pelos dois autores, Victor de Souza Vilela da Silva e Igor Lamoia Queiroz, conforme a divisão de responsabilidades registrada no Apêndice A. Implementado em TypeScript e consumido pela IDE como dependência local, conforme descrito na Seção 4.1, o núcleo encadeia as fases de análise léxica, análise sintática, geração de código intermediário em três

endereços e execução pelo interpretador, recebendo dinamicamente a configuração da linguagem definida pelo usuário (palavras-chave, operadores em forma de palavra, literais booleanos, delimitadores de bloco e terminador de instrução).

Por se tratar da base técnica compartilhada entre os dois volumes, o detalhamento interno de cada fase do núcleo, incluindo as estratégias de varredura léxica, a análise sintática por descida recursiva, a geração de código de três endereços e o funcionamento do interpretador, é objeto do trabalho complementar de autoria de Victor de Souza Vilela da Silva. Este volume concentra-se na forma como a IDE aciona e integra esse núcleo, detalhada nas seções seguintes.

4.3 Customização Dinâmica de Tokens e Sintaxe

A funcionalidade central da plataforma, que consiste na possibilidade de o usuário personalizar dinamicamente a sintaxe e o vocabulário da linguagem, foi materializada em um conjunto coordenado de componentes, *contexts* e utilitários. O elemento central dessa funcionalidade é o `KeywordContext`, um *context* do React que mantém, em memória e na camada de armazenamento local do navegador, o estado completo da customização ativa, incluindo o mapeamento de palavras-chave personalizadas, os operadores em forma de palavra, os literais booleanos, os delimitadores de bloco e o terminador de instrução.

O contrato técnico que sustenta a customização é definido pelo módulo `config.ts` do pacote `compiler`, que declara, por meio de tipos TypeScript, todos os elementos da linguagem passíveis de personalização. O Trecho de Código 4.1 apresenta esses tipos, que funcionam como a interface formal entre a camada de interface, responsável pela coleta das preferências do usuário, e o núcleo do tradutor, responsável pela construção da gramática efetiva no momento da varredura léxica.

```
1 export type KeywordMap = Record<string, number>;
2
3 export type OperatorWordMap = {
4   logical_or?: string;      logical_and?: string;
5   logical_not?: string;     less?: string;
6   less_equal?: string;     greater?: string;
7   greater_equal?: string;  equal_equal?: string;
8   not_equal?: string;
9 };;
```

```

10
11 export type BooleanLiteralMap = {
12     true?: string;
13     false?: string;
14 };
15
16 export type LexerBlockDelimiters = {
17     open: string;
18     close: string;
19 };
20
21 export type LexerConfig = {
22     customKeywords?: KeywordMap;
23     operatorWordMap?: OperatorWordMap;
24     booleanLiteralMap?: BooleanLiteralMap;
25     statementTerminatorLexeme?: string;
26     blockDelimiters?: LexerBlockDelimiters;
27     locale?: string;
28     indentationBlock?: boolean;
29     tabWidth?: number;
30 };

```

Listing 4.1 – Tipos que descrevem a configuração customizável do lexer.

A definição de quais elementos da linguagem seriam exibidos como configuráveis foi guiada pelos achados empíricos de Stefik e Siebert (2013), segundo os quais a sintaxe tradicional derivada da família C atua como uma barreira significativa para estudantes iniciantes. Foram disponibilizadas para customização as palavras-chave de declarações de tipo (`int`, `float`, `bool`, `string`), de controle de fluxo (`if`, `else`, `for`, `while`, `break`, `continue`, `return`, `switch`, `case`), as operações de entrada e saída (`print`, `scan`), os operadores lógicos (`&&`, `||`, `!`) e relacionais (`==`, `!=`, `>`, `>=`, `<`, `<=`), os literais booleanos (`true`, `false`), os delimitadores de bloco (chaves ou indentação) e o terminador de instrução. Essas opções abrangem precisamente os elementos que oneram a carga cognitiva inicial discutidos por Stefik e Siebert (2013), mantendo intactos os aspectos estruturais responsáveis pela consistência das demais fases do compilador.

A consolidação do estado da customização no `KeywordContext` é exposta às demais camadas da aplicação por meio do utilitário `buildLexerConfig`, apresentado no Trecho de Código 4.2. Esse utilitário traduz o estado em memória, contendo os mapeamentos de palavras-chave, os operadores, os literais booleanos, os delimitadores de bloco e o terminador de instrução, em um descritor compatível com o tipo `LexerConfig` do pacote `compiler`, atuando como a fronteira efetiva entre o estado da interface e a invocação do tradutor.

```

1 const buildKeywordMap = useCallback(() : Record<string, number> => {
2   const map: Record<string, number> = {};
3   for (const m of customization.mappings) {
4     map[m.custom] = m.tokenId;
5   }
6   return map;
7 }, [customization.mappings]);
8
9 const buildLexerConfig = useCallback(() : IDECompilerConfigPayload =>
10   {
11     return buildLexerConfigFromCustomization(customization);
12   }, [customization]);

```

Listing 4.2 – Utilitário `buildLexerConfig` do `KeywordContext`, que materializa a configuração corrente em um descritor consumível pelo lexer.

A interface de customização foi implementada como um assistente progressivo no formato *wizard*, materializado no componente `keyword-customizer`, que conduz o usuário por etapas independentes, cada uma responsável por um aspecto específico da personalização. O fluxo é controlado pelo `wizard-model`, que mantém o estado de navegação e valida cada passo antes de permitir o avanço. A declaração formal das etapas do assistente é apresentada no Trecho de Código 4.3, no qual cada elemento do arranjo `WIZARD_STEPS` associa um identificador, um título exibido na barra de progresso, uma descrição textual e um ícone semântico, oferecendo ao usuário um mapa visual contínuo do fluxo de personalização.

```

1 export const WIZARD_STEPS = [
2   { id: "identity", title: "Identidade",
3     description: "Escolha o ponto de partida da linguagem.",
4     icon: "fingerprint" },
5   { id: "IO", title: "Entrada/Saída",

```

```

6      description: "Defina saída, leitura e a linguagem das variáveis."
7      ,
8      icon: "cable" },
9  { id: "types",      title: "Tipagem",
10     description: "Defina leitura, escrita e o vocabulário usado para
11       declarar variáveis.",
12     icon: "book-open-text" },
13  { id: "structure", title: "Estrutura",
14     description: "Configure blocos, delimitadores e o fim das
15       instruções.",
16     icon: "blocks" },
17  { id: "rules",      title: "Operadores",
18     description: "Ajuste os operadores, literais booleanos e regras
19       de decisão.",
20     icon: "sigma" },
21  { id: "flow",       title: "Fluxo",
22     description: "Customize o vocabulário de controle.",
23     icon: "route" },
24  { id: "review",     title: "Revisão",
25     description: "Confira o resumo final antes de salvar.",
26     icon: "clipboard-check" },
27 ] as const;

```

Listing 4.3 – Declaração do arranjo WIZARD_STEPS, que organiza as etapas do assistente de personalização.

A cada alteração proposta pelo usuário, o componente `preview-builder` produz, em tempo real, um trecho de código exemplificativo que reflete a customização corrente, oferecendo uma visualização imediata do impacto das mudanças sobre a aparência dos programas. O Trecho de Código 4.4 ilustra a função `buildDelimiterSnippet`, responsável por interpolar, em um molde fixo, as palavras-chave e os delimitadores efetivamente escolhidos pelo usuário, produzindo um pequeno programa exemplificativo coerente com a configuração corrente.

```

1 export function buildDelimiterSnippet (
2   draft: StoredKeywordCustomization,
3 ): string {
4   const print = getKeyword(draft, "print");

```

```

5  const scan = getKeyword(draft, "scan");
6  const lineEnding = buildLineEnding(draft);
7  const funcao = getKeyword(draft, "funcao");
8  const baseCodeSnippet = `${funcao} main() `;
9
10 const open = draft.blockDelimiters.open.trim() || "{";
11 const close = draft.blockDelimiters.close.trim() || "}";
12
13 return `${baseCodeSnippet}${open}\n\t${print}("Ola Mundo!")${
14     lineEnding
15     ${scan}(nome)${lineEnding}\n\t${print}("Me chamo:", nome)\n${close}
16     `;
17 }

```

Listing 4.4 – Função buildDelimiterSnippet, que gera o *preview* dinâmico exibido durante a personalização.

Essa estratégia de *feedback* contínuo segue as recomendações de Nystrom (2021) para ambientes interativos de aprendizagem, reduzindo a distância cognitiva entre a configuração e o resultado observável.

A validação das customizações é centralizada em um conjunto de funções utilitárias dedicadas, hospedadas no diretório `contexts/keyword/keyword-validator.ts`. A função `validateCustomKeyword` garante que palavras-chave personalizadas atendam às restrições léxicas elementares e não colidam entre si; `validateBlockDelimiters` verifica a consistência dos delimitadores de bloco. Toda a verificação ocorre de forma preventiva, no momento da alteração pelo usuário, impedindo que configurações inválidas sejam persistidas ou propagadas ao analisador léxico, em conformidade com os princípios de detecção precoce de erros discutidos por Cooper e Torczon (2012).

O Trecho de Código 4.5 ilustra a estratégia de verificação aplicada aos delimitadores de bloco. A função encadeia as restrições elementares (ambos os delimitadores preenchidos, conformidade com o formato de identificadores, distinção entre o delimitador de abertura e o de fechamento e ausência de colisão com palavras reservadas), devolvendo, em caso de violação, a mensagem internacionalizada correspondente. Quando todas as restrições são atendidas, a função retorna `null`, sinalizando à interface que a configuração pode ser persistida.

```

1  export function validateBlockDelimiters(

```

```

2   value: BlockDelimiters,
3 ): string | null {
4   const open = value.open.trim();
5   const close = value.close.trim();
6
7   if (!open && !close) return null;
8   if (!open || !close) {
9     return ui.validation_fill_delimiters;
10  }
11
12  if (!WORD_REGEX.test(open) || !WORD_REGEX.test(close)) {
13    return ui.validation_invalid_delimiter_format;
14  }
15
16  if (open === close) {
17    return ui.validation_delimiters_must_differ;
18  }
19
20  if (
21    ORIGINAL_KEYWORDS.includes(open) ||
22    ORIGINAL_KEYWORDS.includes(close)
23  ) {
24    return ui.validation_delimiters_reserved;
25  }
26
27  return null;
28 }

```

Listing 4.5 – Função `validateBlockDelimiters`, responsável pela verificação preventiva dos delimitadores de bloco.

A persistência da customização foi modelada em duas camadas complementares. As configurações ativas no editor são salvas localmente, no *localStorage* do navegador, por meio do utilitário `keyword-language-storage`, garantindo que o estado da personalização sobreviva ao recarregamento da página e à navegação entre rotas da aplicação. Linguagens nomeadas e exportadas pelo usuário, por sua vez, são persistidas no *back-end* via API, viabilizando

o compartilhamento entre dispositivos e a vinculação de linguagens específicas a exercícios oferecidos pelo professor. O utilitário `language-creator-navigation` orquestra as transições entre as etapas de criação, gravação e ativação de linguagens, mantendo coesão entre o estado em memória, o estado no *localStorage* e o estado remoto.

Por fim, o impacto visual imediato da customização sobre o editor Monaco é mediado pela função `updateJavaMMKeywords`. Essa função reconstrói dinamicamente a gramática léxica registrada no editor cada vez que o `KeywordContext` sinaliza uma alteração na customização, refletindo de imediato o realce de sintaxe, a sugestão automática de palavras-chave e a documentação contextual exibida pelo Monaco. O Trecho de Código 4.6 apresenta sua assinatura, evidenciando o desacoplamento entre o módulo de personalização e o módulo de edição: a função recebe a instância do Monaco, o conjunto atualizado de mapeamentos e as opções de linguagem, delegando à função interna `registerJavaMMLanguage` o registro completo do dialeto sob o identificador `java-`. Essa abordagem evita a necessidade de reinicializar o editor a cada mudança e oferece ao usuário uma sensação de continuidade absoluta no ato de personalização.

```
1 export const JAVAMM_LANGUAGE_ID = "java--";
2
3 export type JavaMMKeywordMapping = {
4   original: string;
5   custom: string;
6   tokenId: number;
7 };
8
9 export function updateJavaMMKeywords (
10   monaco: typeof monacoEditor,
11   keywordMappings: JavaMMKeywordMapping[],
12   options: JavaMMLanguageOptions = {},
13 ) {
14   registerJavaMMLanguage(monaco, keywordMappings, options);
15 }
```

Listing 4.6 – Função `updateJavaMMKeywords`, ponto de integração entre o estado da customização e o Monaco Editor.

4.4 Edição de Código e Sistema de Arquivos Virtual

A experiência de edição foi construída em torno do *Microsoft Monaco Editor*, o mesmo motor de edição utilizado pelo Visual Studio Code, integrado à aplicação por meio do pacote `@monaco-editor/loader`. A linguagem personalizada do projeto foi registrada sob o identificador `java-`, e dois temas próprios, `editor-glass-dark` e `editor-glass-light`, foram modelados a partir do módulo `EditorThemes.ts` para harmonizar visualmente o editor com o restante da identidade da plataforma, em ambas as variações de tema claro e escuro.

O gerenciamento do editor foi encapsulado no `EditorContext`, que abriga a referência da instância Monaco, o código-fonte corrente, o nome do arquivo ativo, os marcadores de linha utilizados para destacar erros e o conjunto de configurações de exibição. A interação entre o usuário e o editor foi enriquecida pelo *hook* `useDebugger`, responsável por sinalizar visualmente, durante a execução passo a passo, qual instrução do código intermediário está sendo processada pelo interpretador, em consonância com o requisito pedagógico de tornar visível o processo de tradução.

A persistência local dos arquivos do usuário foi modelada pelo *hook* `useFileSystem`, que implementa um sistema de arquivos virtual sustentado pelo *localStorage* do navegador. Essa abordagem permite que múltiplos arquivos sejam criados, renomeados, removidos e alternados sem qualquer dependência de servidor remoto, mantendo a plataforma plenamente funcional inclusive em cenários de uso *offline*. O *hook* `useEditorWithFileSystem` combina o estado do editor com o sistema de arquivos virtual, garantindo a sincronização bidirecional entre o conteúdo exibido no Monaco e o conteúdo persistido localmente. Complementarmente, os *hooks* `useExplorer` e `useSearch` oferecem, no painel lateral, navegação por estrutura de arquivos e busca textual sobre todos os arquivos do projeto, replicando as funcionalidades essenciais de uma IDE profissional.

4.5 Execução das Análises Léxica e Sintática a partir da IDE

A invocação das análises do compilador a partir da IDE foi modelada por meio de *hooks* dedicados que encapsulam toda a interação entre a camada de interface e o núcleo de tradução. Diferentemente da estratégia adotada por ferramentas como a plataforma Laila, discutida no referencial teórico, na qual o código submetido pelo usuário é encaminhado a um serviço remoto para processamento, neste trabalho a execução das análises foi modelada inteiramente

no navegador, sem intermediação de rotas HTTP. Essa decisão arquitetural foi viabilizada pela organização *monorepo* descrita na Seção 4.1, que permite à IDE consumir o pacote `compiler` como dependência local e instanciar suas classes diretamente no contexto de execução do cliente.

O *hook* `useLexerAnalyse` concentra a orquestração da fase léxica. A cada solicitação do usuário, o *hook* recupera o código-fonte corrente, consulta o `KeywordContext` para obter a configuração ativa da linguagem dinâmica, normaliza o mapeamento efetivo de palavras-chave por meio do utilitário `buildEffectiveKeywordMap` e instancia uma nova classe `Lexer` do pacote `compiler`, passando como parâmetros o mapa de palavras-chave personalizadas, os operadores em forma de palavra, os literais booleanos, o terminador de instrução, os delimitadores de bloco e o *locale* ativo. A varredura é então executada pelo método `scanTokens`, e a coleção de *tokens* resultante, juntamente com eventuais avisos e mensagens informativas coletadas pelo sistema de *issues*, é devolvida diretamente ao componente que originou a invocação, sem qualquer ida ao servidor.

O Trecho de Código 4.7 evidencia essa instanciação direta. Observa-se a importação do símbolo `Lexer` a partir do pacote local `@ts-compiler-for-java/compiler`, viabilizada pela organização em *monorepo*, e a construção do analisador com a configuração derivada do `KeywordContext`, sem qualquer chamada HTTP intermediária.

```
1 import { Lexer } from "@ts-compiler-for-java/compiler/src/lexer";
2
3 async function runLexerAnalyse(
4   input: RunLexerAnalyseInput,
5 ): Promise<TLexerAnalyseData> {
6   try {
7     const effectiveKeywordMap = buildEffectiveKeywordMap(input.
8       keywordMap);
9     const lexer = new Lexer(input.sourceCode, {
10       customKeywords: effectiveKeywordMap,
11       operatorWordMap: input.operatorWordMap,
12       booleanLiteralMap: input.booleanLiteralMap,
13       statementTerminatorLexeme: input.statementTerminatorLexeme,
14       blockDelimiters: input.blockDelimiters,
15       locale: input.locale,
16       indentationBlock: input.indentationBlock,
```

```

16     });
17     const tokens = lexer.scanTokens();
18     return {
19         message: "Lexical Analysis completed",
20         tokens,
21         warnings: lexer.warnings,
22         infos: lexer.infos,
23         error: null,
24     };
25 } catch (error) {
26     if (error instanceof IssueError) throw error;
27     throw new Error((error as Error).message || "Codigo nao suportado
28         ");
29 }

```

Listing 4.7 – Função `runLexerAnalyse`, que instancia o lexer diretamente no cliente.

Os *tokens* obtidos por essa execução local são renderizados na IDE como cartões interativos pelo componente `token-card`, oferecendo ao usuário uma visualização direta do resultado da fase léxica, incluindo o lexema original, sua categoria e sua posição no código-fonte. Essa exposição direta atende ao propósito pedagógico de tornar visíveis as etapas internas do processo de tradução, característica diferencial da plataforma em relação às ferramentas analisadas no referencial teórico. Eventuais erros lançados pelo lexer, encapsulados pela classe `IssueError`, são interceptados pelo *hook* e materializados tanto no terminal quanto no editor, por meio dos utilitários `showLineIssues` e do mecanismo de marcadores discutido na Seção 4.6.

O *hook* `useIntermediatorCode` segue uma abordagem análoga. Após reaproveitar a tabela de *tokens* produzida pela fase léxica, o *hook* instancia um `TokenIterator` e aciona o analisador sintático diretamente no cliente, integrado a um `Emitter` responsável pela geração das instruções de código intermediário em três endereços. A lista de instruções resultante é igualmente exibida na IDE em um painel dedicado, permitindo ao estudante inspecionar a tradução do código-fonte para uma forma de representação intermediária. Essa execução também ocorre integralmente no navegador, sem qualquer requisição HTTP intermediária.

A escolha por executar as análises léxica e sintática diretamente no cliente, em vez de delegá-las ao servidor por meio de rotas internas do Next.js, atende a três objetivos correlacionados. Primeiro, minimiza a latência percebida pelo usuário, ao eliminar o ciclo de requisição e resposta exigido por uma arquitetura cliente-servidor convencional. Segundo, permite que a plataforma permaneça plenamente funcional mesmo na ausência de conectividade com o servidor de persistência, sustentando os recursos essenciais de edição, análise e interpretação. Terceiro, simplifica a manutenção do código, uma vez que a lógica do compilador é consumida como dependência direta, sem necessidade de empacotamento, serialização ou conversão de tipos entre cliente e servidor. As rotas HTTP do servidor Next.js, situadas sob o caminho `/api`, são reservadas exclusivamente ao subsistema de validação automatizada de submissões e à intermediação das chamadas ao *back-end* FastAPI, conforme detalhado na Seção 4.7.

Por fim, o *hook* `use-api-queries` consolida, por meio do *TanStack React Query*, todas as consultas relacionadas a turmas, exercícios, listas de exercícios e submissões, oferecendo cache automático, revalidação em segundo plano e tratamento padronizado de erros para toda a aplicação. Essas consultas, distintamente das análises locais discutidas anteriormente, são as únicas que envolvem comunicação remota efetiva com o serviço de persistência, mantendo claras as fronteiras entre a execução local do compilador e o consumo de recursos do Ambiente Virtual de Aprendizagem.

4.6 Execução do Código Intermediário e *Feedback* no Terminal

O componente `terminal`, hospedado em `components/terminal`, foi modelado como o canal primário de *feedback* interativo entre o programa em execução e o usuário. Sua arquitetura interna é composta por três peças: o componente `body`, que renderiza o histórico de saídas e o prompt de entrada; o componente `header`, que apresenta os controles de execução e o estado corrente; e o *hook* `useKeyboardShortcuts`, que mapeia atalhos teclados às operações de limpeza, foco e interrupção da execução. O terminal é controlado pelo `TerminalContext`, responsável por gerenciar seu estado de abertura, fechamento e visibilidade na composição da tela.

A execução dos programas foi modelada de modo que as operações `print` e `scan` produzidas pelo interpretador sejam diretamente conectadas ao terminal embutido. O `Interpreter`, instanciado no servidor Next.js, recebe funções de retorno (`stdout` e `stdin`) cujo comportamento varia conforme o modo de execução: em modo interativo, as funções são associadas ao terminal

da IDE; em modo de validação automatizada de exercícios, são conectadas a *buffers* controlados que capturam a saída para comparação com o resultado esperado. Esse desacoplamento entre o motor de execução e a camada de apresentação atende aos princípios de baixo acoplamento discutidos por Martin (2019) e foi inspirado nas abstrações de fluxos de entrada e saída descritas por Cooper e Torczon (2012).

O sistema de *feedback* de erros foi modelado em duas categorias complementares. Os erros estáticos detectados durante a compilação, como falhas léxicas, sintáticas e semânticas, são reportados pelo módulo `issue` do pacote `compiler`, que classifica cada ocorrência como `IssueError` (impede a tradução), `IssueWarning` (alerta não fatal) ou `IssueInfo` (informação contextual). Essas mensagens são interceptadas pela IDE e materializadas tanto no terminal, em formato textual, quanto no editor, por meio de marcadores de linha sublinhados, em conformidade com as práticas de relato de erros discutidas por Aho, Sethi e Ullman (1995). Os erros dinâmicos, detectados pelo interpretador em tempo de execução (como divisões por zero, conversões impossíveis durante a leitura de entrada ou acessos a posições inválidas de *arrays*), são encapsulados pela classe `RuntimeError` e gerenciados pelo `RuntimeErrorContext`, que coordena a exibição contextualizada no terminal e o destaque visual no editor.

Para garantir que mensagens de erro sejam pedagogicamente úteis, mesmo quando a linguagem corrente está fortemente personalizada, todas as mensagens passam pelo módulo de internacionalização `i18n` do pacote `compiler`, que oferece traduções para os idiomas *pt-BR*, *pt-PT*, *es* e *en*. Adicionalmente, as mensagens são adaptadas ao vocabulário corrente da customização: termos como “palavra reservada esperada” são substituídos pela palavra-chave efetivamente esperada na configuração ativa, evitando a exposição ao usuário de termos canônicos que tenham sido propositalmente ocultados pela personalização.

4.7 Sistema de Correção e Avaliação Automatizada

A plataforma incorpora um subsistema de avaliação automatizada inspirado nos juízes *online* discutidos por Ferreira, Silva e Ribeiro (2025) e Garcia (2024), materializado pela rota `/api/submissions/validate`. A modelagem do fluxo de submissão de exercícios partiu da estrutura conceitual desses juízes, mas foi adaptada ao contexto da linguagem dinâmica: enquanto os juízes tradicionais exigem que o aluno se adapte à sintaxe rígida de uma linguagem comercial, a plataforma proposta permite que cada exercício carregue sua própria configuração da linguagem, podendo, inclusive, restringir o aluno a um vocabulário específico definido pelo

professor.

O modelo de dados que sustenta o Ambiente Virtual de Aprendizagem foi estruturado em torno das entidades `User`, `Class`, `Exercise`, `ExerciseList`, `Submission` e `LanguageImage`. Essa modelagem permite que professores criem turmas, vinculem exercícios estruturados em listas, definam casos de teste com entradas e saídas esperadas, e associem a cada exercício uma “imagem de linguagem”, isto é, uma configuração da linguagem dinâmica que será imposta ao aluno no momento da resolução. As consultas e mutações sobre essas entidades são tratadas pelo *back-end* FastAPI, com persistência em SQLAlchemy assíncrono, e gerenciadas no *front-end* pelo *TanStack React Query*, que cuida do cache e da revalidação automática.

4.8 Acessibilidade, Responsividade e Finalização de Fluxos

A finalização dos fluxos da plataforma foi conduzida com especial atenção aos requisitos de acessibilidade e responsividade. A escolha das bibliotecas *shadcn/ui* e *Radix UI* como base dos componentes primitivos, incluindo diálogos, menus, *tooltips*, alertas, abas e *popovers*, garante, por padrão, o cumprimento das diretrizes WAI-ARIA, com suporte nativo à navegação por teclado, à leitura por tecnologias assistivas e ao gerenciamento correto de foco. Essa abordagem alinha-se aos princípios de design inclusivo discutidos por Stefik e Siebert (2013) e Ladner (2019) na concepção da linguagem Quorum, segundo os quais a acessibilidade não pode ser uma camada adicional, mas deve estar incorporada à fundação dos componentes utilizados.

A responsividade da plataforma foi modelada pelo *hook* `useBreakpoint`, que monitora as dimensões da janela e expõe, de forma reativa, qual *breakpoint* Tailwind está ativo. As composições de tela, hospedadas no diretório `views`, foram concebidas para reorganizar dinamicamente seus painéis conforme a faixa de resolução: em telas amplas, o editor, o painel lateral e o terminal são apresentados simultaneamente; em telas estreitas, esses componentes assumem disposições alternativas, com a possibilidade de alternância controlada pelo usuário. Essa adaptação foi sustentada pelas utilidades de responsividade do Tailwind CSS v4 e pela função utilitária `cn`, que compõe classes condicionais com a biblioteca *tailwind-merge*.

O tratamento de temas claro e escuro foi delegado à biblioteca *next-themes*, integrada ao `ThemeContext`, que mantém o tema corrente sincronizado entre o sistema operacional do usuário, a preferência salva e os múltiplos componentes da aplicação. Os temas do editor Monaco são alternados de forma coordenada, garantindo consistência visual entre o ambiente de edição e o restante da interface. As notificações ao usuário, por sua vez, são gerenciadas

pelo `ToastContext`, oferecendo uma fila padronizada de mensagens com diferentes níveis de severidade.

A internacionalização da plataforma foi implementada por meio do mecanismo nativo do Next.js, com suporte aos locais *pt-BR* (padrão), *pt-PT*, *es* e *en*, organizados em arquivos JSON sob o diretório `locales`. Todas as cadeias visíveis ao usuário foram externalizadas para esses arquivos, permitindo que a plataforma seja oferecida em diferentes idiomas e que o conteúdo pedagógico se ajuste ao público-alvo. Essa decisão atende à premissa central do projeto, segundo a qual barreiras linguísticas devem ser sistematicamente reduzidas no contexto da aprendizagem introdutória de programação.

A autenticação de usuários foi modelada por um fluxo simplificado, sustentado pelo `AuthContext` no *front-end* e pelas rotas `/api/auth/login`, `/api/auth/register` e `/api/auth/me`. As credenciais são propagadas entre as camadas por meio de *cookies* HTTP gerenciados pelo utilitário `auth-cookies`, e a navegação protegida é controlada pelos *layouts* compartilhados, que verificam a presença de uma sessão válida antes de renderizar páginas restritas. Essa estratégia atende aos requisitos mínimos de segregação entre os papéis de aluno, professor e administrador, e mantém a complexidade do modelo de autenticação compatível com o caráter educacional do projeto.

4.9 Apresentação das Telas da Plataforma

Com o intuito de tornar tangíveis as decisões arquiteturais, os *contexts* de estado, os *hooks* e os componentes discutidos ao longo das seções anteriores, esta seção apresenta as principais telas efetivamente implementadas até o encerramento desta etapa do trabalho. A organização segue o fluxo natural de uso da plataforma: parte-se da identidade visual da página inicial, percorrem-se os painéis dos perfis de aluno e de professor e, por fim, ilustra-se o passo a passo de criação e personalização de uma linguagem dinâmica diretamente na IDE.

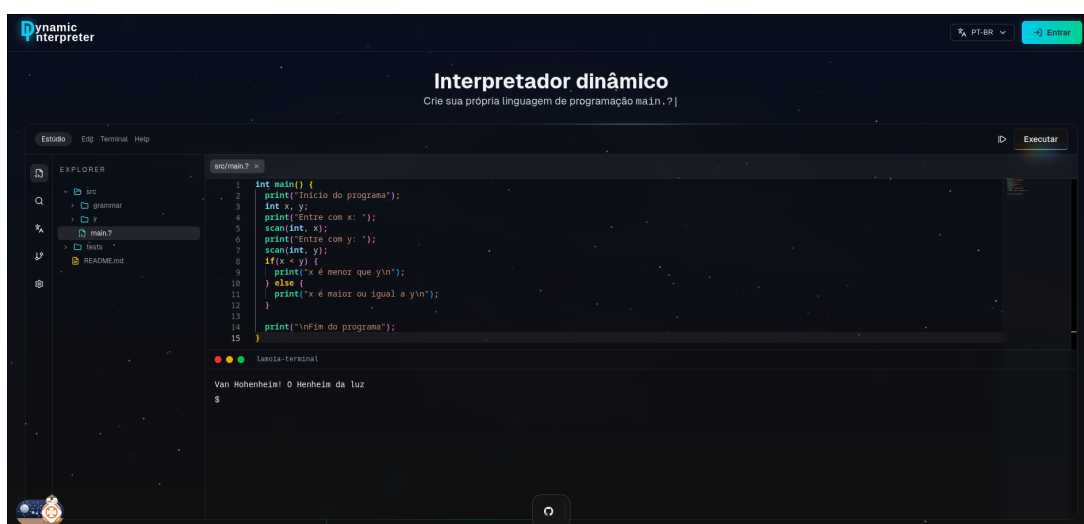
Página Inicial e Identidade Visual

A página inicial materializa a identidade visual da plataforma e funciona como porta de entrada para os fluxos de autenticação, navegação e divulgação dos recursos disponíveis. A composição segue o sistema de temas claro e escuro discutido na Seção 4.8, com troca controlada pelo `ThemeContext`, e foi modelada com componentes da biblioteca *shadcn/ui* para garantir consistência tipográfica, contraste adequado e respeito às diretrizes de acessibilidade WAI-ARIA.

As Figuras 4.2a e 4.2b apresentam, lado a lado, as duas variações de tema, evidenciando o cuidado com a paridade visual entre as versões clara e escura.



(a) Tema claro.



(b) Tema escuro.

Figura 4.2 – Página inicial da plataforma em suas duas variações de tema.

Fonte: elaborado pelo autor.

Painel do Aluno

O painel do aluno consolida, em uma única tela, todas as turmas em que o estudante está matriculado, as listas de exercícios pendentes e o histórico recente de submissões. A tela foi modelada para reduzir o número de cliques necessários até o exercício seguinte, oferecendo cartões interativos que conduzem o aluno diretamente à composição da IDE com a linguagem dinâmica já configurada pelo professor. A Figura 4.3 ilustra a disposição geral desse painel.

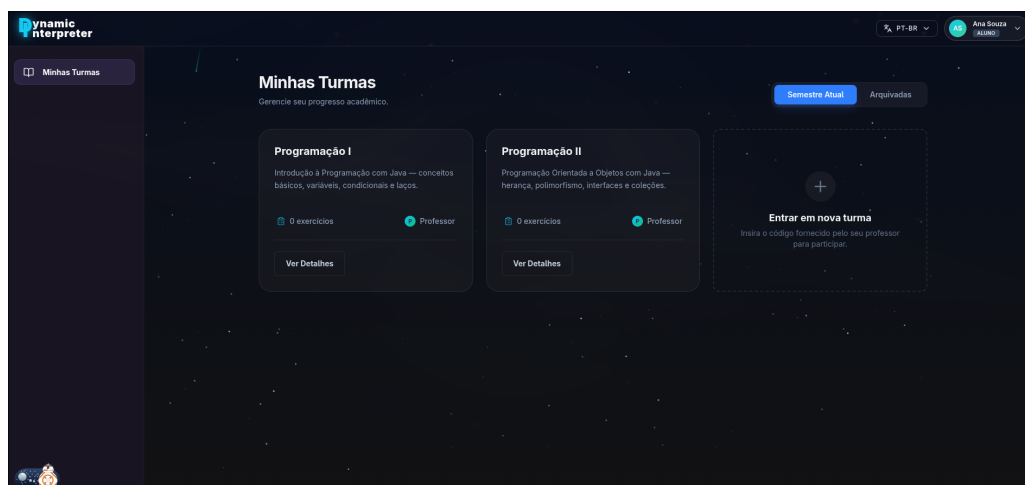
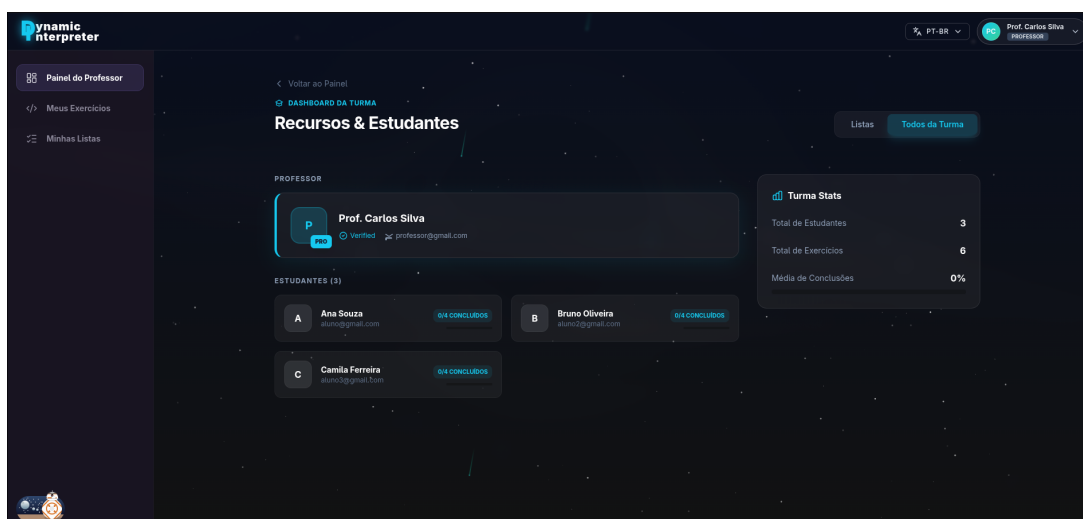


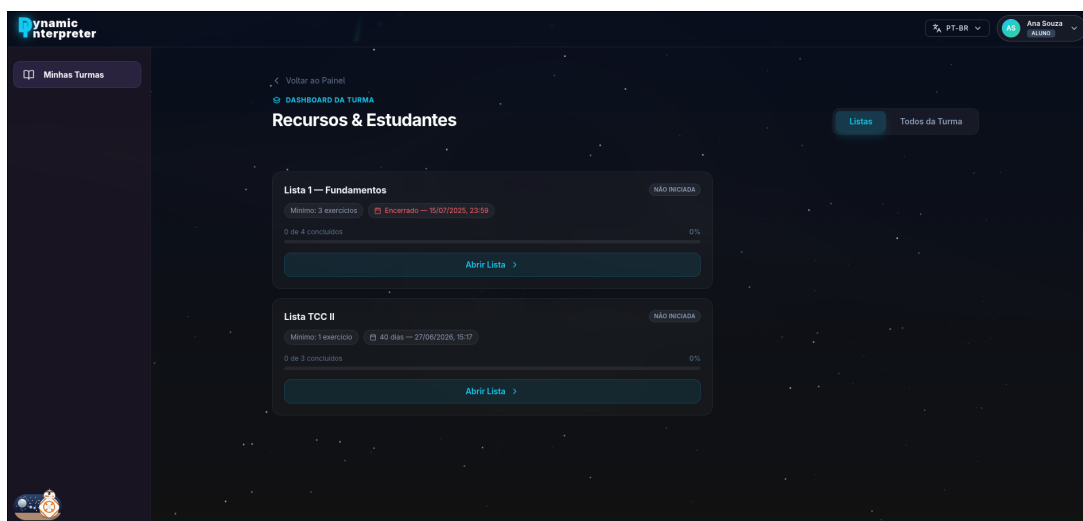
Figura 4.3 – Painel principal do aluno, com acesso às turmas, listas e exercícios.

Fonte: elaborado pelo autor.

Ao selecionar uma turma específica, o aluno é conduzido à tela de detalhamento, na qual visualiza informações da disciplina, listas publicadas pelo professor e seu progresso individual em cada exercício. A Figura 4.4a apresenta essa visão, enquanto a Figura 4.4b detalha a estrutura interna de uma lista de exercícios, na qual cada item exibe o respectivo estado (não iniciado, em andamento ou submetido) e a configuração da linguagem dinâmica vinculada ao enunciado.



(a) Detalhamento de uma turma do aluno.



(b) Lista de exercícios sob a ótica do aluno.

Figura 4.4 – Composições da experiência do aluno: visão da turma e visão da lista de exercícios.

Fonte: elaborado pelo autor.

Painel do Professor

O painel do professor segue a mesma metáfora de organização do painel do aluno, mas amplia o conjunto de ações disponíveis, incorporando criação e gestão de turmas, listas de exercícios e enunciados. A Figura 4.5 apresenta a visão consolidada do professor, com acesso direto às turmas sob sua responsabilidade. Ao selecionar uma turma, são exibidas as listas associadas, conforme a Figura 4.6a, e o conjunto de listas pode ser inspecionado em maior detalhe pela visão dedicada da Figura 4.6b.

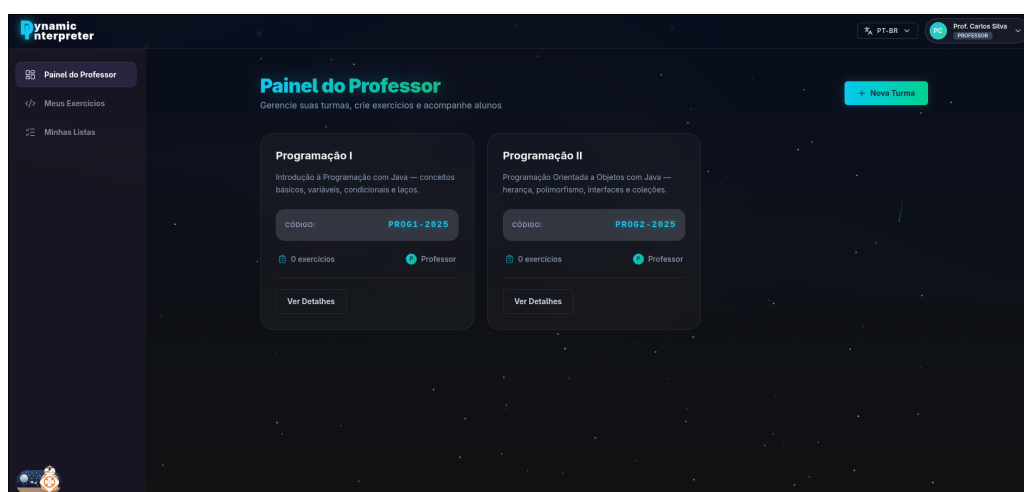
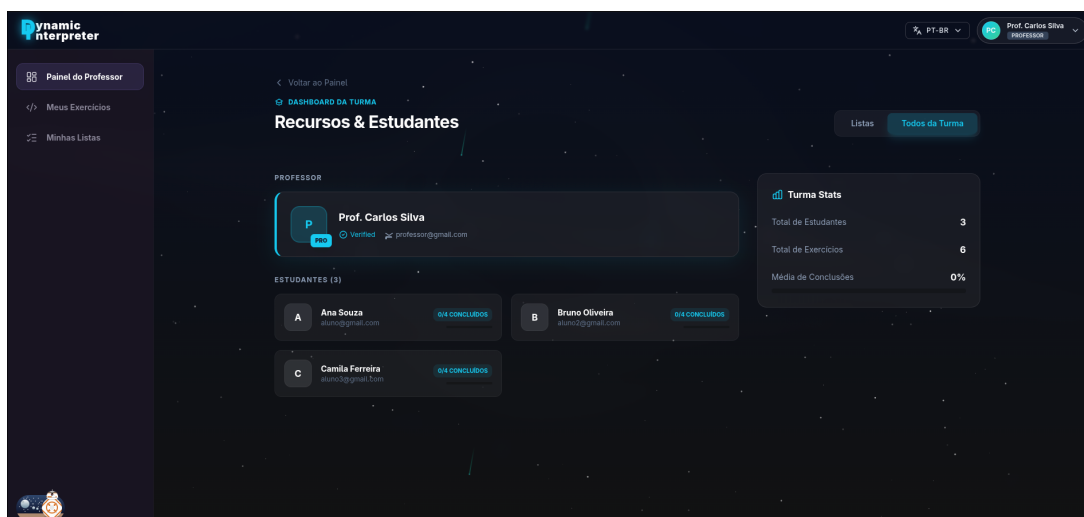
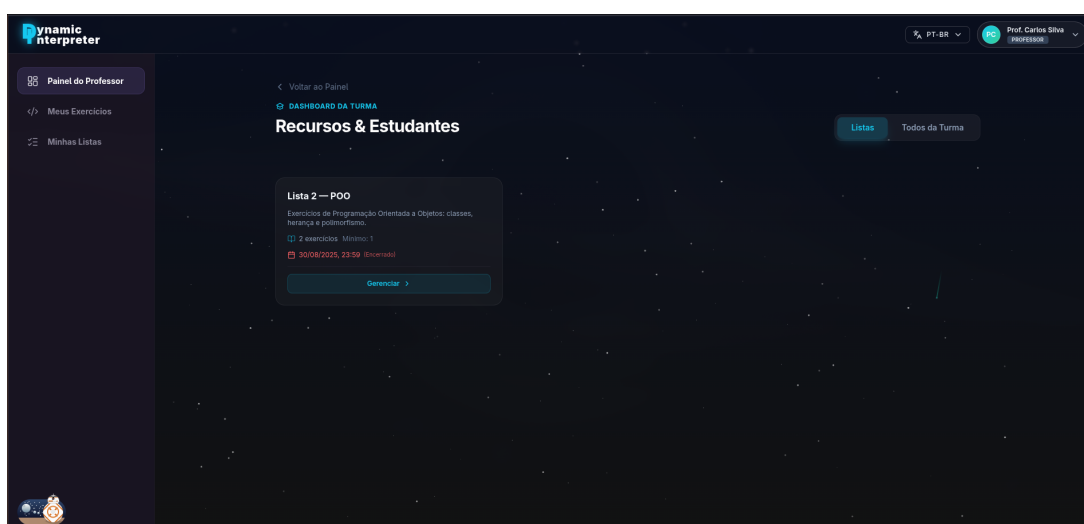


Figura 4.5 – Painel principal do professor.

Fonte: elaborado pelo autor.



(a) Detalhamento de uma turma do professor.



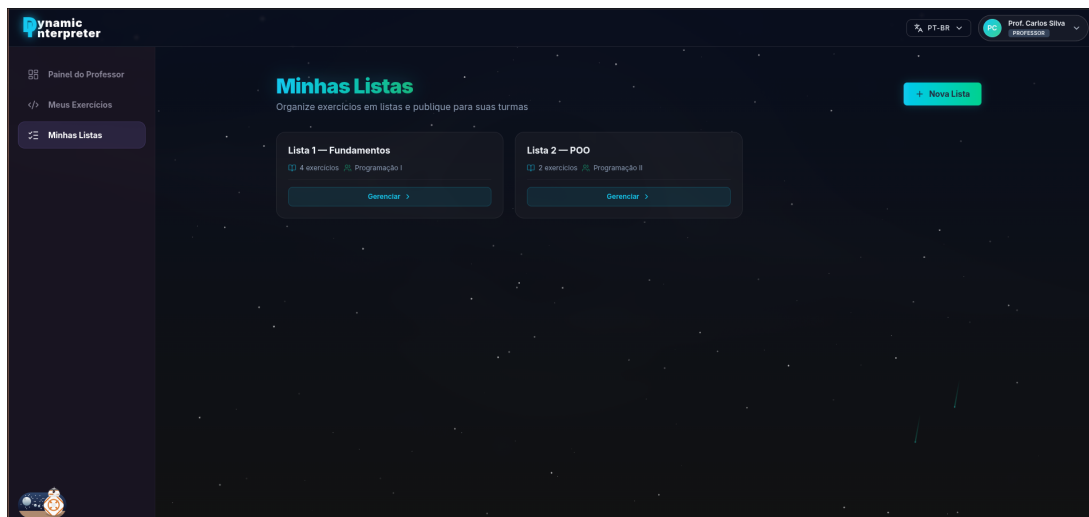
(b) Listas de exercícios vinculadas à turma.

Figura 4.6 – Composições da experiência do professor: visão da turma e visão das listas associadas.

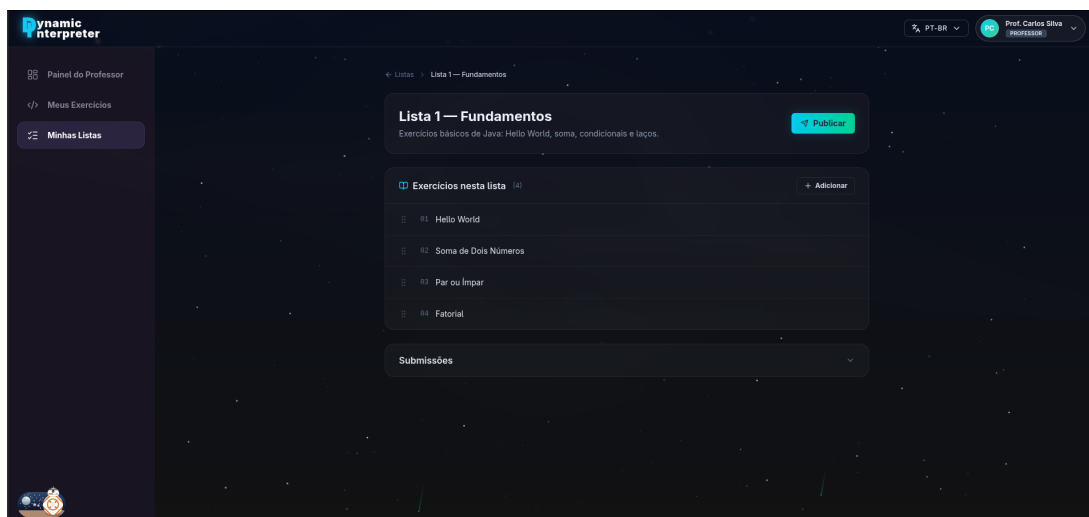
Fonte: elaborado pelo autor.

Gerenciamento de Listas e Exercícios

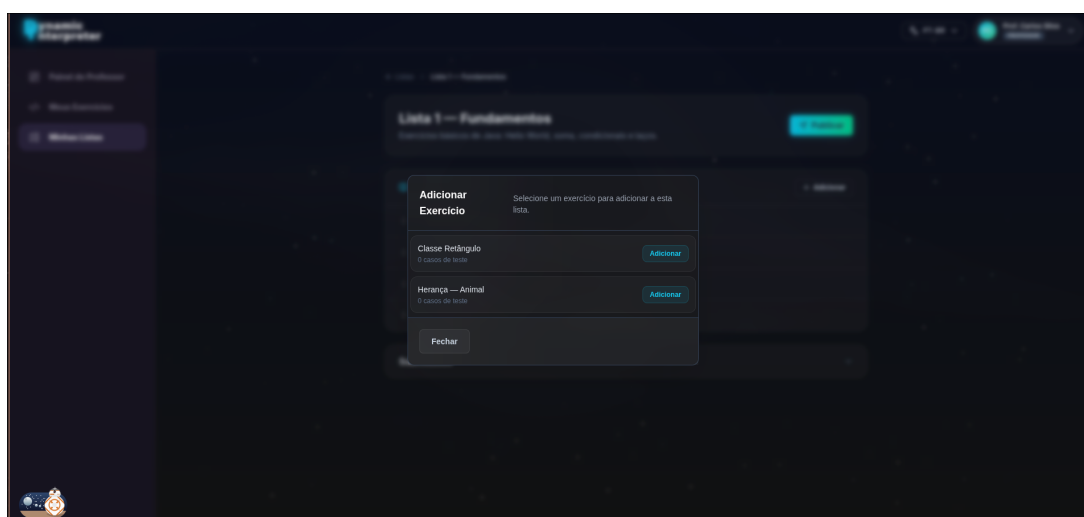
A criação e a manutenção de listas pelo professor ocorrem em telas dedicadas que isolam cada operação em um fluxo independente. A Figura 4.7 reúne, em uma única composição, as quatro etapas centrais desse processo: a visualização das listas existentes, a configuração dos parâmetros gerais da lista (título, descrição e prazo), a vinculação de exercícios à lista por meio do utilitário de busca e seleção, e a publicação da lista para a turma escolhida.



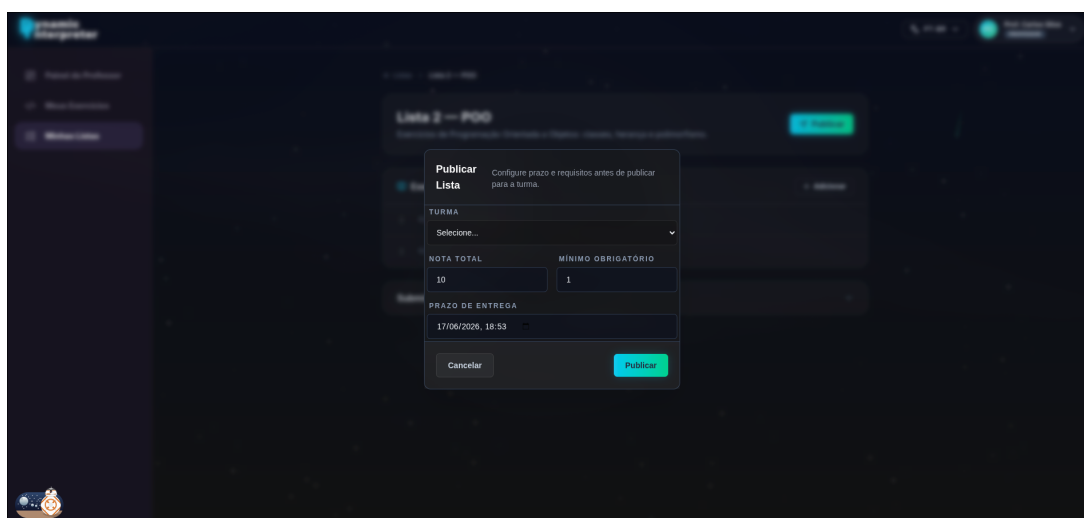
(a) Listagem das listas existentes.



(b) Configuração dos parâmetros gerais.



(c) Vinculação de exercícios à lista.

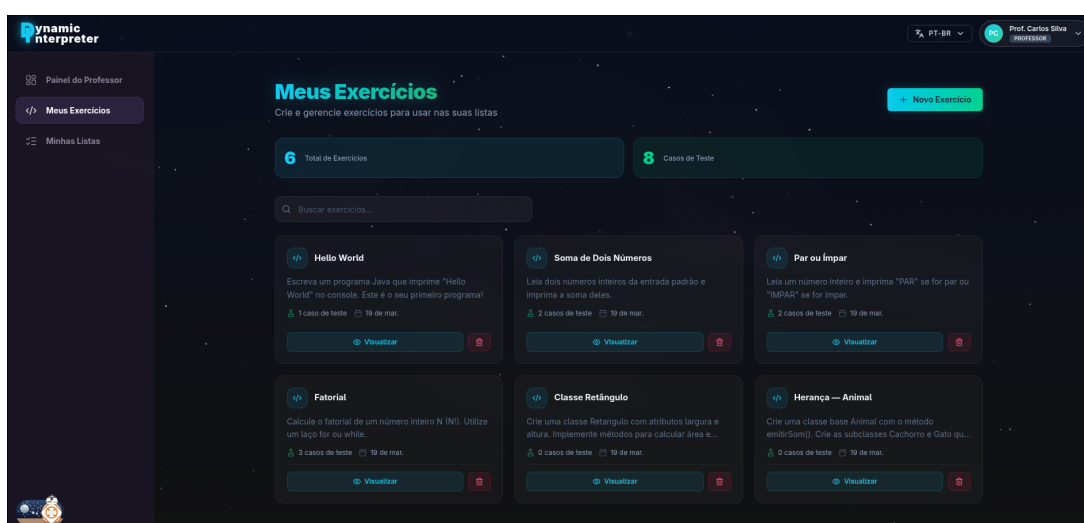


(d) Publicação da lista para a turma.

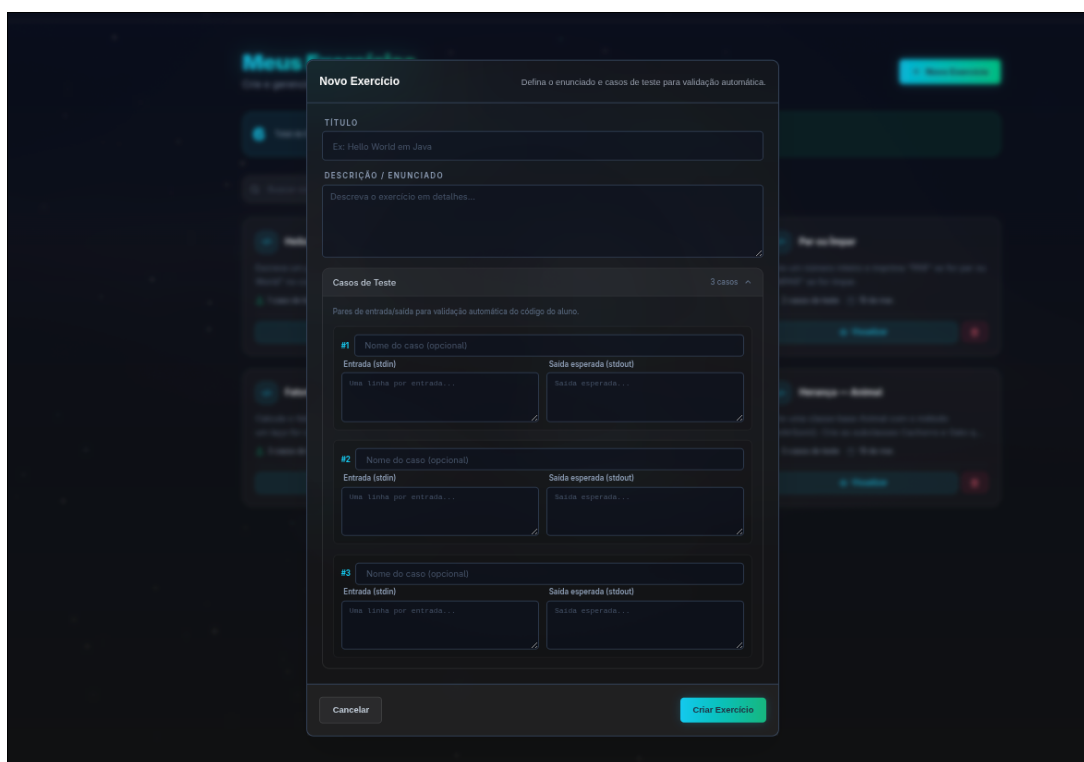
Figura 4.7 – Fluxo de criação e gestão de listas de exercícios pelo professor.

Fonte: elaborado pelo autor.

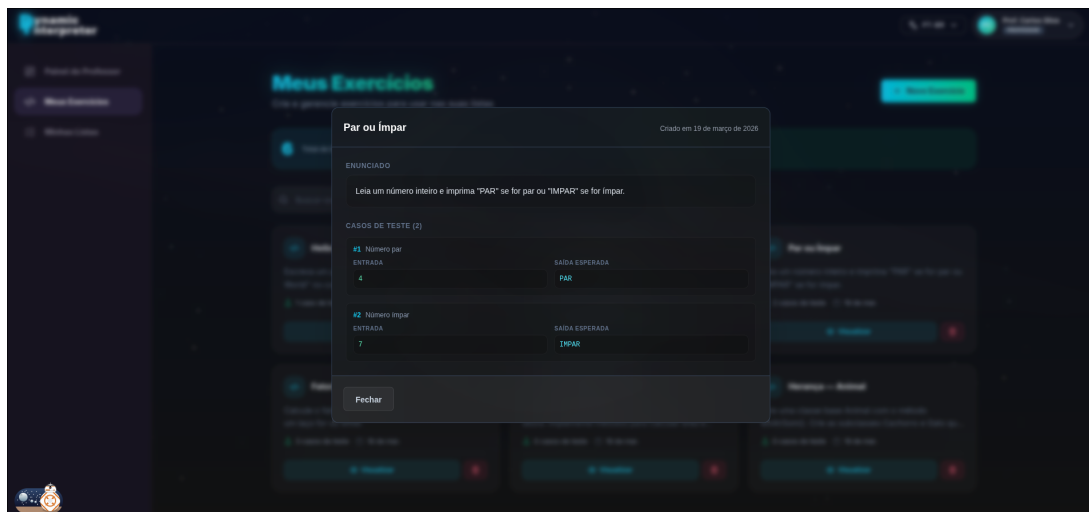
A gestão dos enunciados individuais ocorre em um conjunto análogo de telas, ilustrado pela Figura 4.8. A tela de listagem permite a busca, a filtragem e a ordenação dos enunciados; a tela de criação centraliza, em um único formulário, a descrição do problema, os casos de teste com entradas e saídas esperadas e a vinculação a uma imagem de linguagem que delimita o vocabulário disponível ao aluno; a tela de detalhamento mostra a composição final do exercício; e a tela de submissões consolida todas as resoluções enviadas pelos alunos, com os respectivos vereditos do subsistema de avaliação automatizada discutido na Seção 4.7.



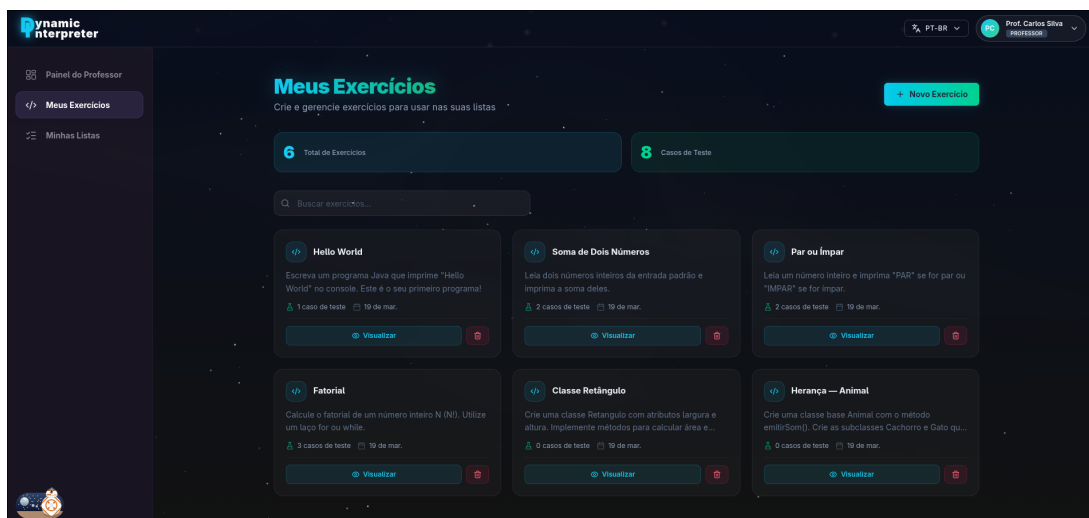
(a) Listagem dos enunciados existentes.



(b) Criação de um novo enunciado.



(c) Detalhamento de um exercício.



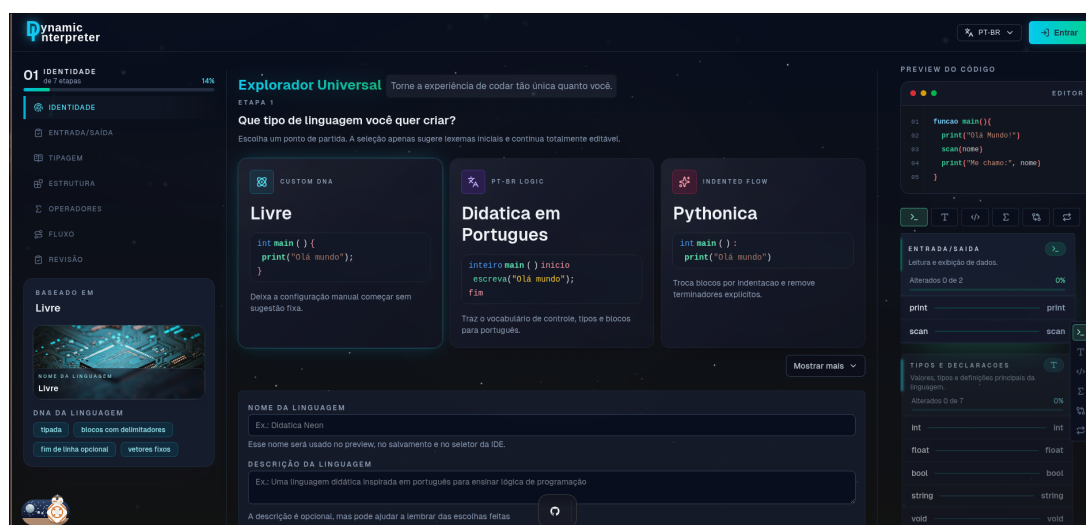
(d) Submissões dos alunos para o exercício.

Figura 4.8 – Fluxo de criação e acompanhamento dos enunciados pelo professor.

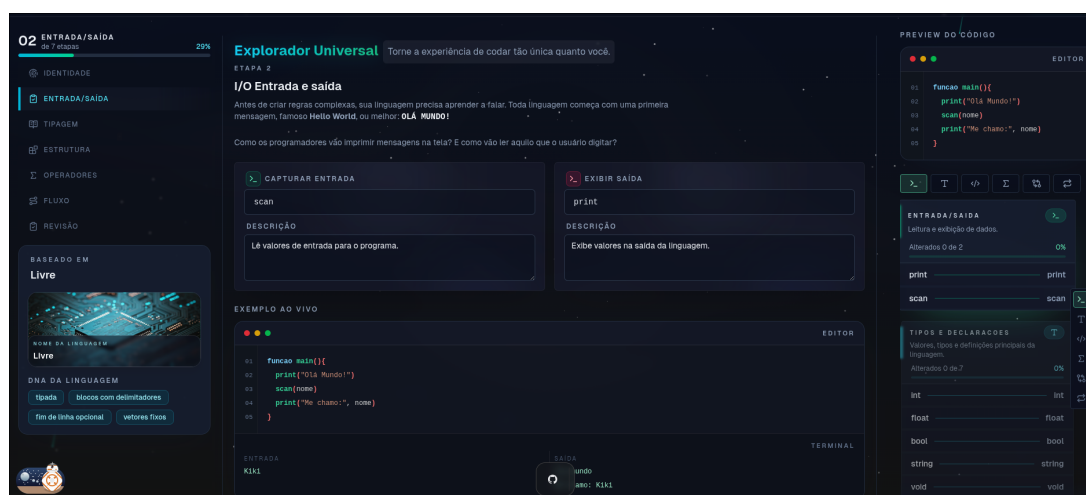
Fonte: elaborado pelo autor.

4.9.1 Fluxo de Personalização da Linguagem Dinâmica

A funcionalidade central da plataforma, que corresponde à personalização dinâmica do vocabulário e das regras estruturais da linguagem, foi materializada em um assistente progressivo composto por seis etapas sequenciais. Esse *wizard*, discutido na Seção 4.3, conduz o usuário por escolhas independentes, validadas isoladamente, com geração de *preview* em tempo real do código resultante. As figuras a seguir documentam, em ordem, cada uma das etapas efetivamente implementadas.



(a) Etapa 1: definição do nome da linguagem e da identidade da customização.



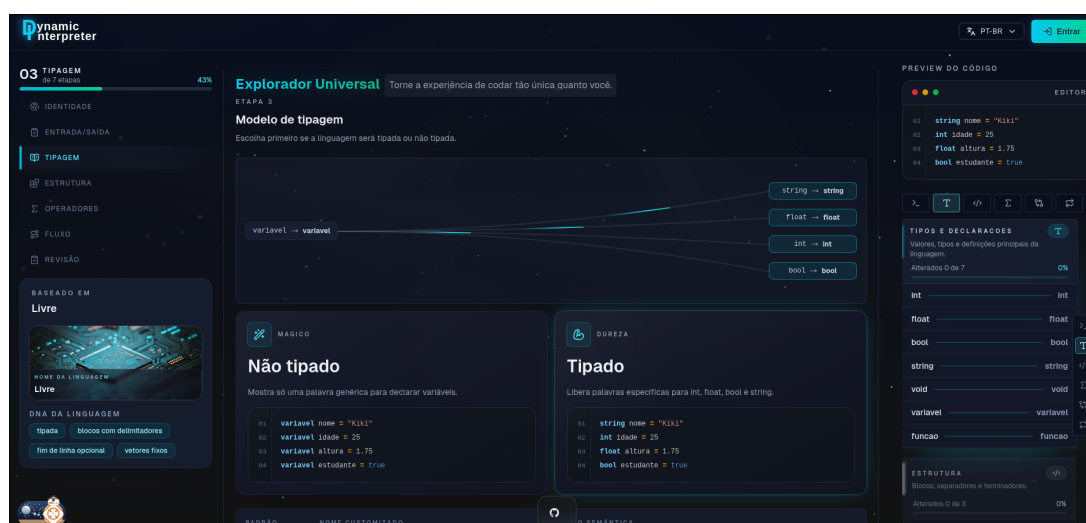
(b) Etapa 2: configuração das palavras-chave de tipos primitivos.

Figura 4.9 – Etapas 1 e 2 do assistente de personalização da linguagem dinâmica.

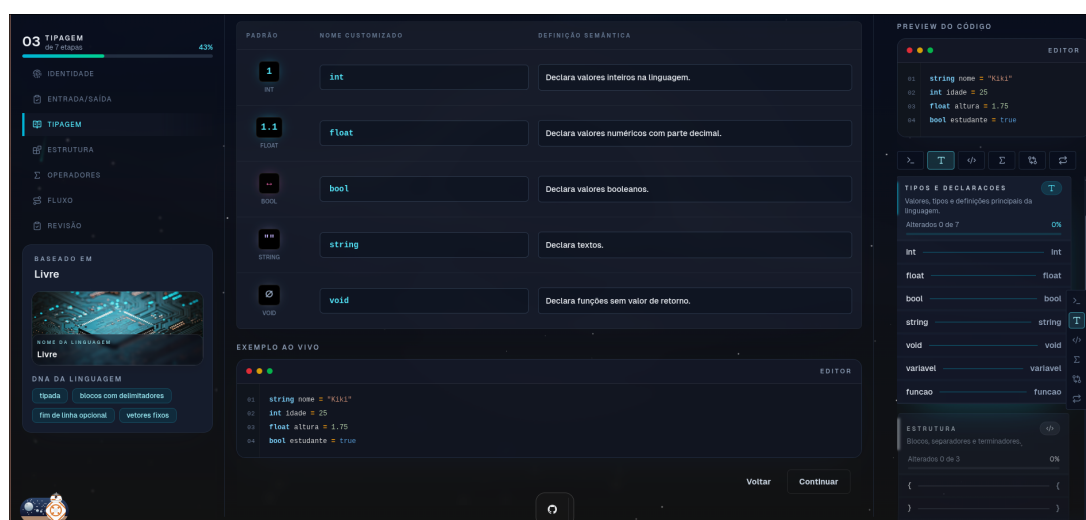
Fonte: elaborado pelo autor.

A terceira etapa concentra-se na personalização das palavras-chave de controle de fluxo, oferecendo ao usuário a possibilidade de redefinir comandos como `if`, `else`, `for`, `while`, `break`, `continue` e `return`. A Figura 4.10 apresenta as duas variações dessa

etapa, contemplando a configuração das palavras-chave básicas e a configuração estendida com construtores como `switch` e `case`.



(a) Etapa 3.1: palavras-chave básicas de controle de fluxo.

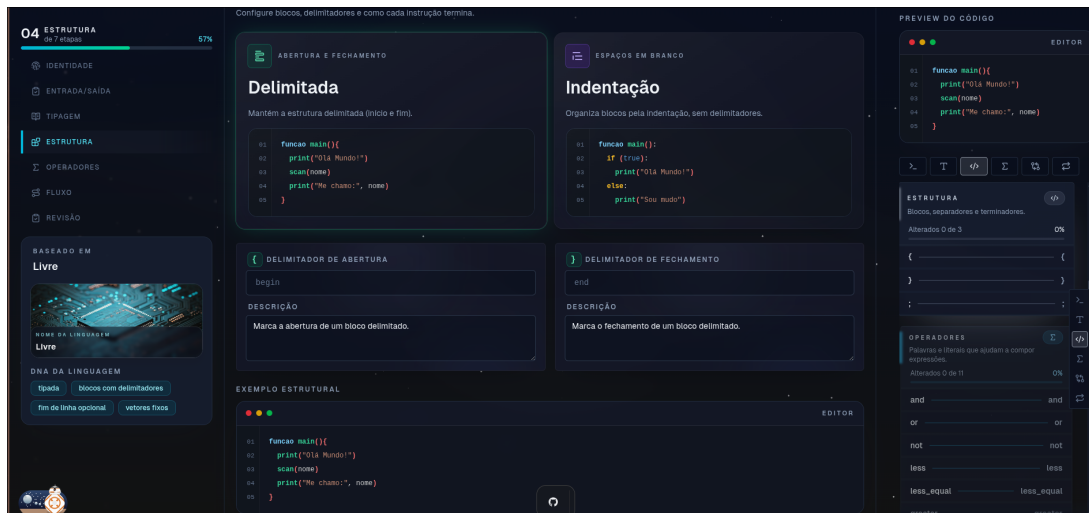


(b) Etapa 3.2: construtores estendidos de controle de fluxo.

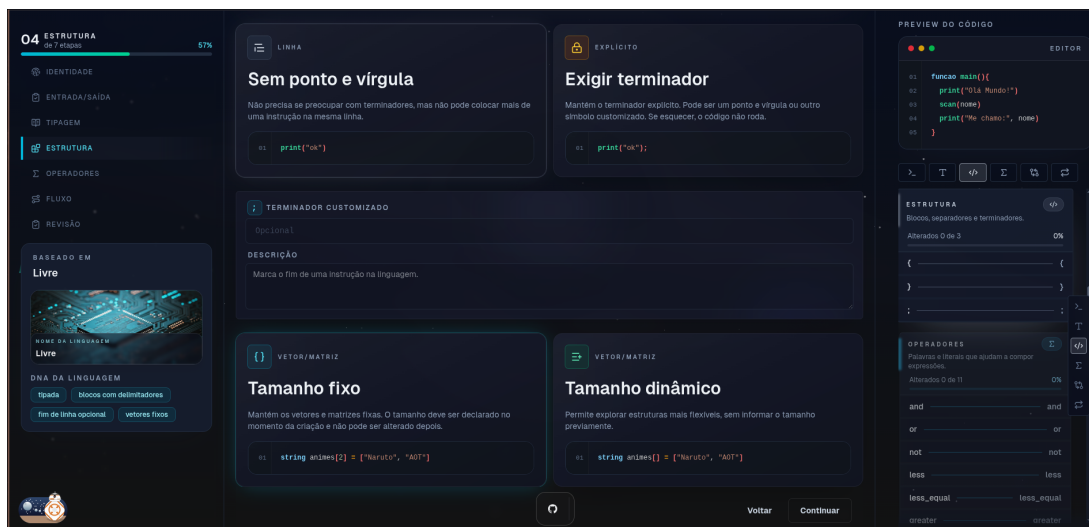
Figura 4.10 – Etapa 3 do assistente, dedicada às palavras-chave de controle de fluxo.

Fonte: elaborado pelo autor.

A quarta etapa habilita a substituição de operadores lógicos e relacionais por palavras de uso natural, em consonância com os achados empíricos de Stefik e Siebert (2013) sobre o impacto de símbolos obscuros na carga cognitiva inicial. A Figura 4.11 ilustra as duas variações dessa etapa, contemplando, primeiro, a definição de operadores lógicos como `e`, `ou` e `não e`, em seguida, a redefinição dos operadores relacionais por meio do mesmo princípio de palavras significativas.



(a) Etapa 4.1: operadores lógicos em forma de palavra.



(b) Etapa 4.2: operadores relacionais em forma de palavra.

Figura 4.11 – Etapa 4 do assistente, dedicada à substituição de operadores por palavras.

Fonte: elaborado pelo autor.

A quinta etapa, apresentada na Figura 4.12, é responsável pela configuração dos literais booleanos, oferecendo ao usuário a oportunidade de adotar termos como verdadeiro e falso em substituição aos canônicos `true` e `false`. A finalização do assistente ocorre na etapa seis, na qual o usuário define os delimitadores de bloco (chaves, palavras-chave ou indentação) e o terminador de instrução, conforme ilustrado pela Figura 4.13, que reúne as três variações dessa etapa: configuração dos delimitadores, do terminador e da pré-visualização consolidada da linguagem personalizada.

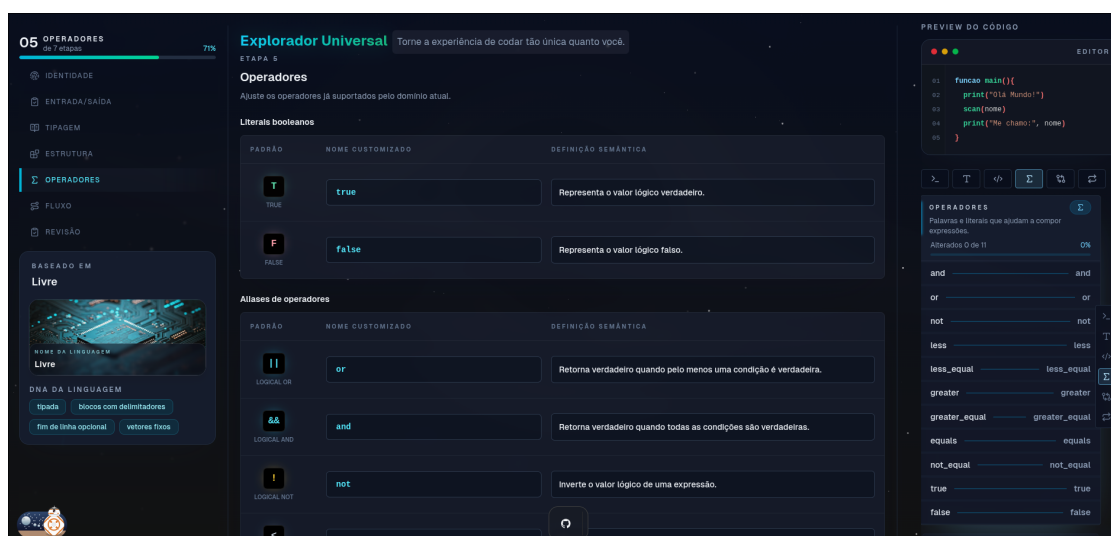
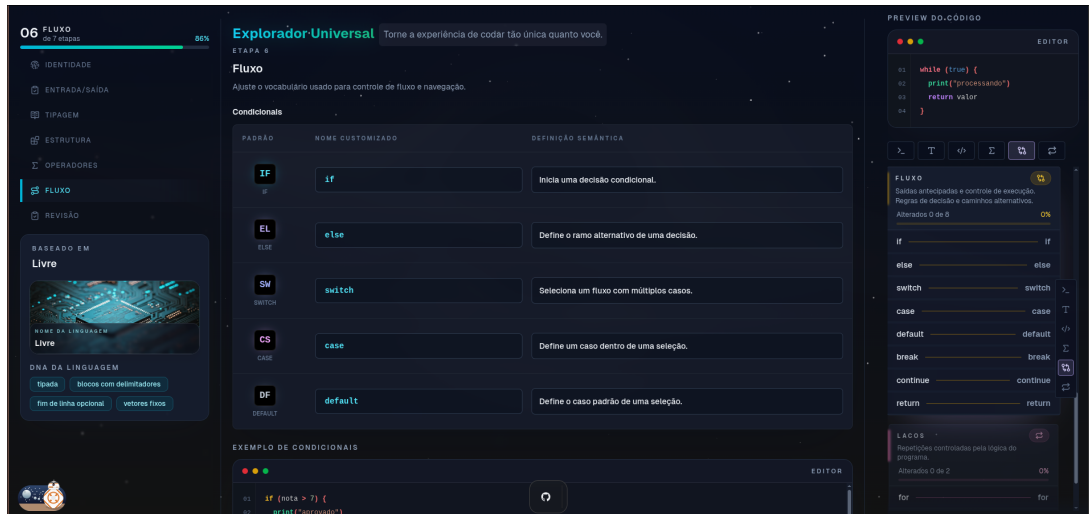


Figura 4.12 – Etapa 5: personalização dos literais booleanos.

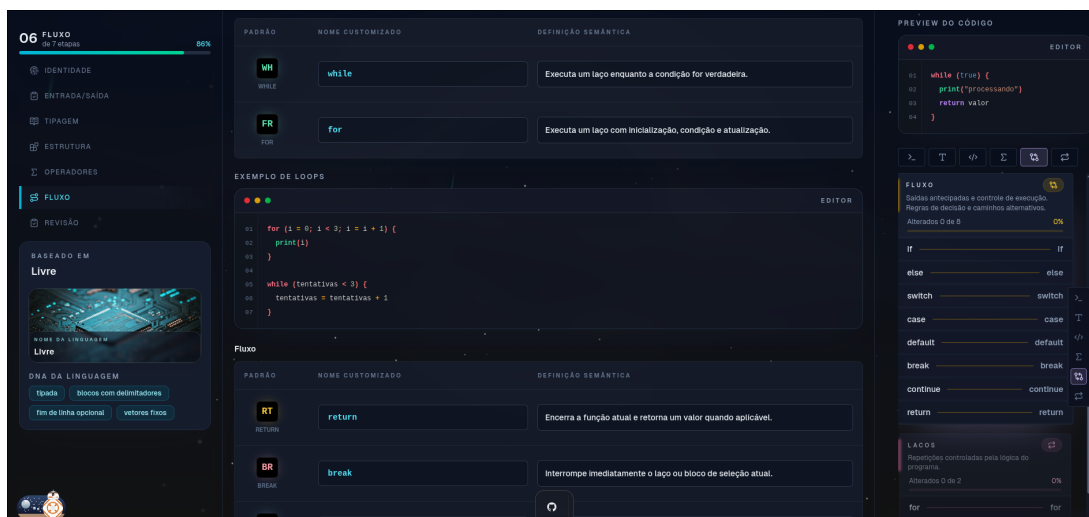
Fonte: elaborado pelo autor.



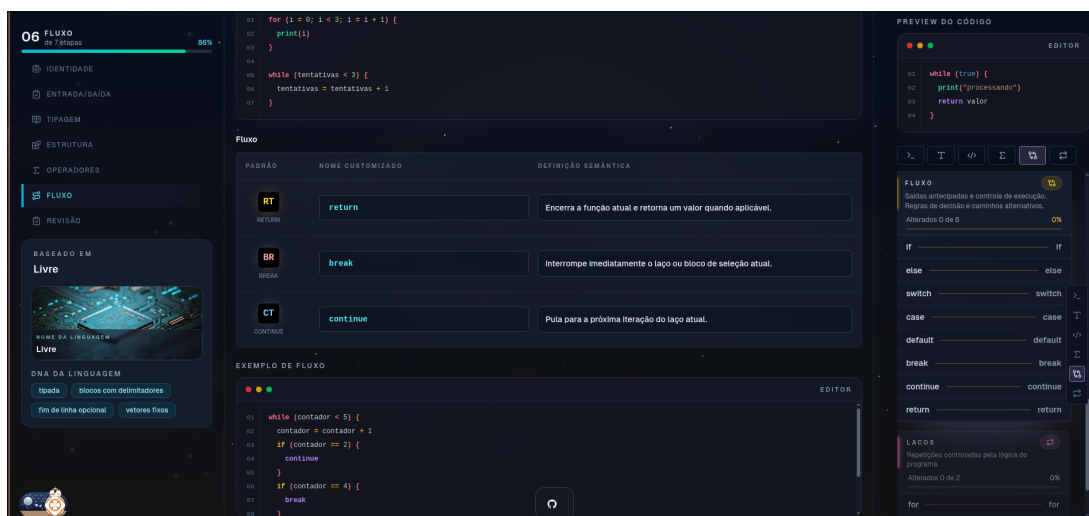
Remover este espaço em branco



(a) Delimitadores de bloco.



(b) Terminador de instrução.



(c) Pré-visualização final.

Figura 4.13 – Etapa 6 do assistente, dedicada aos delimitadores, ao terminador de instrução e à pré-visualização da linguagem personalizada.

Fonte: elaborado pelo autor.

O conjunto de telas apresentado consolida, sob o ponto de vista visual, as decisões arquiteturais e de modelagem discutidas ao longo deste capítulo. A organização das informações, a coerência entre os perfis de aluno e professor, a granularidade do assistente de personalização e o cuidado com a paridade entre os temas claro e escuro evidenciam o esforço consciente em alinhar a experiência de uso da plataforma aos princípios de UX e de design inclusivo discutidos no referencial teórico.

4.10 Estratégia de Testes Automatizados

A validação contínua da plataforma foi sustentada por uma suíte de testes automatizados construída com o *framework* Vitest, complementada por *jsdom* para simulação do ambiente de navegador em testes que envolvem componentes React. A organização dos testes refletiu a topologia interna do projeto: testes unitários acompanham diretamente os módulos correspondentes em diretórios `__tests__`, enquanto testes de integração foram concentrados em arquivos com sufixos `.spec.ts` ou `.spec.tsx` próximos aos componentes que validam.

A adoção do TDD discutido por Aniche (2012) foi particularmente útil em construções críticas, como o validador de palavras-chave personalizadas, o construtor de *preview* interativo da customização, o adaptador do grafo de gramática e o *pipeline* de validação de submissões. Nesses pontos, pequenas alterações na lógica poderiam comprometer fluxos inteiros da plataforma, e a presença de testes automatizados garantiu a estabilidade do comportamento mesmo diante de refatorações estruturais frequentes. Em conformidade com a pirâmide de testes discutida por Martin (2019), a maior parte da suíte concentra-se no nível unitário, garantindo execuções rápidas e *feedback* imediato, enquanto os testes de integração verificam o comportamento agregado entre componentes, *contexts* e *hooks*.

Os mesmos testes são executados localmente durante o desenvolvimento e também acionados pelo *pipeline* de integração contínua antes da incorporação de qualquer mudança ao ramo principal do repositório, assegurando que as características essenciais da plataforma, como a consistência da customização ativa entre recarregamentos, a sincronização correta entre o editor e o sistema de arquivos virtual, a fidelidade do *feedback* de erros ao usuário e a correção dos resultados das submissões, sejam preservadas ao longo de toda a evolução incremental do projeto.



REFERÊNCIAS

ADAPTWORKS. **Agile: Desenvolvimento de Software com Entregas Frequentes e Foco no Valor de Negócio**. São Paulo: Casa do Código, 2012.

AHO, A. V.; SETHI, R.; ULLMAN, J. D. **Compiladores: princípios, técnicas e ferramentas**. Rio de Janeiro: LTC, 1995. Tradução de Daniel de Ariosto Pinto.

ALCANTARA, F. **Tabela de Símbolos**. 2024. Disponível em: <<https://frankalcantara.com/lf/11-tabelaSimbolos.html>>.

ANICHE, M. **Test-Driven Development: Teste e Design no Mundo Real**. São Paulo: Casa do Código, 2012.

BENEDETTI, T. Como escolher a melhor plataforma educativa em 2025. **Blog TutorMundi**, 2025. Disponível em: <<https://tutormundi.com/blog/>>. Acesso em: 2026.

COOPER, K. D.; TORCZON, L. **Engineering a Compiler**. 2. ed. Waltham: Elsevier / Morgan Kaufmann, 2012.

FASTAPI. **Recursos e Documentação Oficial do FastAPI**. 2026. Disponível em: <<https://fastapi.tiangolo.com/>>. Acesso em: 07 abr. 2026.

FERREIRA, B.; SILVA, C. L.; RIBEIRO, F. d. C. L. Um estudo de caso da ferramenta beecrowd como tecnologia da informação e comunicação na aprendizagem de algoritmos. **ForScience**, v. 13, n. 2, 2025. Disponível em: <<https://doi.org/10.29069/forscience.2025v13n2.e1313>>.

GARCIA, L. **Beecrowd: como desenvolver habilidades praticando programação**. Alura, 2024. Disponível em: <<https://www.alura.com.br/artigos/beecrowd-como-desenvolver-habilidades-praticando-programacao>>. Acesso em: 07 abr. 2026.

GOMES, D. **O que é Portugol Studio?** DevMedia, 2020. Disponível em: <<https://www.devmedia.com.br/o-que-e-portugol-studio/40764>>. Acesso em: 07 abr. 2026.

LADNER, R. E. **Learn About the New Quorum Studio**. Computer Science Teachers Association (CSTA), 2019. Disponível em: <<https://csteachers.org/learn-about-the-new-quorum-studio/>>. Acesso em: 07 abr. 2026.

LAI, J. et al. **Next.js 16**. [S.l.]: Vercel, 2025.

LIMA, G. de. **Arquitetura da informação: aprenda a organizar conteúdos**. Alura, 2024. Disponível em: <<https://www.alura.com.br/artigos/arquitetura-da-informacao>>.

MÃO NO CÓDIGO. **BEECROWD: Todo PROGRAMADOR deveria conhecer | BEECROWD (antigo Uri Online Judge)**. 2022. Vídeo do YouTube. Disponível em: <<https://www.youtube.com/watch?v=ExemploUrl>>. Acesso em: 07 abr. 2026.

MAKERKIT. **Next.js Server Actions: The Complete Guide (2026)**. 2026.

MARTIN, R. C. **Código Limpo: Habilidades Práticas do Agile Software**. Rio de Janeiro: Alta Books, 2009.

MARTIN, R. C. **The Clean Coder: A Code of Conduct for Professional Programmers**. Upper Saddle River: Prentice Hall, 2011.

MARTIN, R. C. **Arquitetura Limpa: O Guia do Artesão para Estrutura e Design de Software**. Rio de Janeiro: Alta Books, 2019.

MARTINS, C. **Portugol Studio: Guia Completo de Como Usar a Plataforma**. HostGator, 2025. Disponível em: <<https://www.hostgator.com.br/blog/portugol-studio/>>. Acesso em: 07 abr. 2026.

Mind Makers. 10 tendências em edtech para 2026: o que a escola precisa saber. **Blog Mind Makers**, 2025. Disponível em: <<https://mindmakers.com.br/blog/>>. Acesso em: 2026.

NYSTROM, R. **Crafting Interpreters**. [S.l.]: Genever Benning, 2021. ISBN 978-0990582939.

PRICE, A. M. A.; TOSCANI, S. S. **Implementação de Linguagens de Programação: Compiladores**. Porto Alegre: Sagra Luzzatto, 2001. (~~Série Livros Didáticos~~).

QUORUM. **The Quorum Programming Language: Documentation and Curriculum**. 2025. Disponível em: <<https://quorumlanguage.com/>>. Acesso em: 07 abr. 2026.

RAPOSO, A. B. **Avaliação Heurística: Introdução à Interação Humano-Computador (IHC)**. Rio de Janeiro, RJ, 2012.

SABBAGH, R. **Scrum: Gestão ágil para projetos de sucesso**. São Paulo: Casa do Código, 2014.

SEYFFERT, P. **UX e UI: entenda o que são, as diferenças e o impacto no resultado do seu produto digital**. SoftDesign, 2026. Disponível em: <<https://www.softdesign.com.br/blog/desvendando-a-diferenca-entre-ux-e-ui-design/>>.

SILVA, D. d. S. et al. Minerando dados de um juiz on-line para prever a evasão de estudantes em disciplinas introdutórias de programação. In: **Anais do XXXI Simpósio Brasileiro de Informática na Educação (SBIE)**. Porto Alegre: Sociedade Brasileira de Computação, 2020. p. 1343–1352.

SILVA, M. L. d. **Laila: Uma Plataforma Web para Auxílio ao Ensino de Compiladores**. Monografia (Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação)) — Instituto Federal de Educação, Ciência e Tecnologia do Ceará, Aracati, 2021.

SILVEIRA, P. et al. **Introdução à arquitetura e design de software**. São Paulo: Casa do Código, 2012.

SOUZA, D. M.; BATISTA, M. H. d. S.; BARBOSA, E. F. Problemas e dificuldades no ensino e na aprendizagem de programação: um mapeamento sistemático. **Revista Brasileira de Informática na Educação**, v. 24, n. 1, p. 39–52, 2016.

STEFIK, A.; SIEBERT, S. An empirical investigation into programming language syntax. **ACM Transactions on Computing Education**, ACM, v. 13, n. 4, 2013.

UEYAMA, R. **chibicc: A small C compiler**. 2020. Disponível em: <<https://github.com/rui314/chibicc>>. Acesso em: 07 abr. 2026.

UNIVALI. **Portugol Studio - LITE**. Laboratório de Inovação Tecnológica na Educação, 2026. Disponível em: <<https://univali-lite.github.io/Portugol-Studio/>>. Acesso em: 07 abr. 2026.

VERCEL. **Rendering: Edge and Node.js Runtimes.** [S.l.]: Vercel, 2024.

YABLONSKI, J. **Leis da Psicologia Aplicadas a UX: Usando psicologia para projetar produtos e serviços melhores.** São Paulo, SP: Novatec Editora, 2020.

ZIEMANN, K. Next.js app router vs page router comparison. **Pagepro**, 2025. Disponível em: <https://pagepro.co/blog/nextjs-app-router-vs-pages-router/>.

A APÊNDICE A – PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES

Este apêndice detalha a divisão de responsabilidades entre os dois autores, conforme a pré-proposta aprovada e o plano de trabalho entregue à Coordenação do Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina.

A.1 Atividades Desenvolvidas em Regime Colaborativo

As atividades listadas a seguir foram conduzidas de forma conjunta por Igor Lamoia Queiroz e Victor de Souza Vilela da Silva, com participação efetiva de ambos os integrantes nas etapas de especificação, implementação, integração e revisão:

- Revisão bibliográfica sobre interpretadores, compiladores e ferramentas educacionais relacionadas;
- Definição dos requisitos funcionais e não funcionais e da arquitetura geral do sistema;
- Participação na implementação e validação do núcleo do compilador, incluindo análise léxica, análise sintática, geração e execução de código intermediário;
- Especificação dos mecanismos de injeção dinâmica de regras gramaticais e integração desses mecanismos à experiência de personalização da IDE;
- Integração do núcleo do interpretador com a plataforma web e definição dos contratos de comunicação entre os módulos;
- Revisão mútua de código e tomada de decisões arquiteturais nas etapas de refatoração e evolução do sistema.

A.2 Eixos de Aprofundamento Individual

A partir da base técnica construída em conjunto, cada autor concentrou seu aprofundamento acadêmico em eixos específicos, conforme estabelecido na pré-proposta.

A.2.1 Igor Lamoia Queiroz (este documento)

- Detalhamento da arquitetura da aplicação web (Next.js, Pages Router) e da integração com o Monaco Editor para edição de código com realce de sintaxe configurável;

- Documentação da customização dinâmica de *tokens* e sintaxe por meio do assistente de personalização da linguagem (*wizard*), incluindo o modelo de etapas, a lógica de validação preventiva e o mecanismo de pré-visualização em tempo real;
- Análise da experiência de uso da plataforma, dos contextos React que governam o estado da aplicação e das mediações pedagógicas oferecidas pela interface ao estudante iniciante;
- Descrição da execução das análises léxica e sintática com o código produzido na IDE e da apresentação do código intermediário no terminal integrado;
- Estudos de caso centrados na usabilidade, no engajamento dos estudantes e no *feedback* coletado em ambiente educacional.

A.2.2 Victor de Souza Vilela da Silva (documento complementar)

- Detalhamento das fases de análise léxica e do sistema de varredura por escâneres especializados (*factory pattern*);
- Descrição aprofundada da análise sintática por descida recursiva e da correspondência entre as produções do analisador e a gramática formal da linguagem Java--;
- Documentação da geração de código de três endereços: instrução a instrução, tabela de temporários e tabela de rótulos;
- Análise do funcionamento do interpretador, da tabela de símbolos e do despacho das chamadas de sistema de entrada e saída;
- Desenvolvimento do *back-end* da plataforma em FastAPI, incluindo a modelagem das entidades de domínio, a implementação das rotas REST, a persistência com SQLAlchemy assíncrono e o suporte a autenticação, turmas, exercícios, listas e submissões;
- Estudos de caso centrados na validade técnica do núcleo de tradução e no comportamento do interpretador frente a programas de teste elaborados para cobrir as construções da linguagem.